

ML — Unit 1

UNIT 1 — MASTER NOTEBOOK

Master Study Material

ML Unit 1 — Master Study Material

UNIT 1 — MASTER NOTEBOOK

[Introduction, Performance Metrics, Classification with Decision Trees and KNN](#)

[Table of Contents](#)

[1. What is Machine Learning?](#)

[2. Machine Learning Models](#)

[3. Concept Learning and Inductive Bias](#)

[4. The Find-S Algorithm](#)

[5. Bias–Variance Decomposition](#)

[6. Decision Tree Learning and ID3](#)

[7. ID3: Hypothesis Space, Search, and Inductive Bias](#)

[8. Issues in Decision Tree Learning](#)

[9. Continuous-Valued Attributes and Decision Boundaries](#)

[10. Logistic Regression](#)

[11. Instance-Based Learning — k-Nearest Neighbours](#)

[12. Performance Metrics](#)

[13. ROC and AUC](#)

[14. Practice set](#)

[15. Formula sheet](#)

[16. Most likely exam questions](#)

UNIT 1 — MASTER NOTEBOOK

Introduction, Performance Metrics, Classification with Decision Trees and KNN

UE23CS352A — Machine Learning · PES University · Sem 5 CSE *Official PESIMENS Master Study Material* · 25% of portion · 14 hours

How to use this notebook

1. Read a section top to bottom once; do not skip the worked example.
 2. Redo every worked numerical on paper with the book shut.
 3. The **Exam priority** line under each heading tells you where to spend time.
 4. Do the Practice set (Section 14) before you read its answers.
 5. The Formula sheet (Section 15) is the only page you need the night before.
-

Table of Contents

#	Section	Highest priority inside
1	What is Machine Learning?	(T, P, E) definition — High
2	Machine Learning Models	Comparing the paradigms — Medium
3	Concept Learning and Inductive Bias	Counting formulas — High
4	The Find-S Algorithm	EnjoySport trace — High
5	Bias–Variance Decomposition	Diagnosis and decomposition numerical — High
6	Decision Tree Learning and ID3	Play Tennis trace — High
7	ID3: Hypothesis Space, Search, Inductive Bias	Preference vs restriction bias — High
8	Issues in Decision Tree Learning	Overfitting definition — High
9	Continuous Attributes and Decision Boundaries	Threshold selection — High
10	Logistic Regression	Decision boundary, cost function — High
11	k-Nearest Neighbours	Distance numericals — High
12	Performance Metrics	Confusion matrix to all metrics — High
13	ROC and AUC	Curve interpretation — Medium
14	Practice set — 10 questions with worked answers	—
15	Formula sheet	—
16	Most likely exam questions	—

1. What is Machine Learning?

1.1 Learned rules, and the project pipeline

Exam priority: Low — framing plus a one-line recall item; no slide marks either as a question type.

A traditional program is a set of rules *you* wrote. Machine learning inverts this: you hand the machine **data** (inputs with correct outputs) and it produces the **rules**. You write the *learner*; the data writes the logic.

Analogy. You do not define “dog” for a child; you point at dogs and non-dogs until the child extracts the rule. The rule is the *hypothesis*; the pointing is the *training data*.

The seven steps of a project:

1 Data collection	2 Data preparation	3 Choosing a model
4 Training	5 Evaluation	6 Parameter tuning
7 Prediction		

The usual **train : test split is 80 : 20** — 80% to learn from, 20% held back to measure performance honestly.

Understand: ML learns rules from data. **Memorize:** the seven steps, 80:20. **Practice:** nothing.

1.2 Mitchell’s definition and the (T, P, E) triple

Exam priority: High — the definition is given verbatim with six fully worked T/P/E examples; it is the standard opening question.

“Machine Learning is the study of algorithms that improve their performance **P at some task **T** with experience **E**.”** — Tom Mitchell

A **well-posed learning problem** is the triple **(T, P, E)**: **T** is the task the program must *do*, **P** the number that must improve, **E** the training data it learns from.

The trap: students swap T and P. **T is a verb** (“classify emails”), **P is a number** (“fraction correctly classified”), **E is a dataset**.

#	Problem	Task (T)	Experience (E)	Performance (P)
1	Checkers	Playing checkers	Practice games against itself	Fraction of games won
2	Handwriting recognition	Recognising handwritten words in images	Database of labelled handwritten words	Fraction of words correctly identified
3	Self-driving car	Driving using a vision sensor	Images and steering commands recorded from a human driver	Average distance travelled before an error
4	Text categorization	Assigning a document to its category	Database of pre-classified documents	Fraction of documents correctly tagged
5	Spam detection	Classifying email as spam or not	Database of labelled emails	Fraction of emails correctly classified
6	Credit card fraud	Labelling a transaction fraud or not	Historical labelled transactions	Accuracy, with a higher penalty when fraud is labelled not-fraud

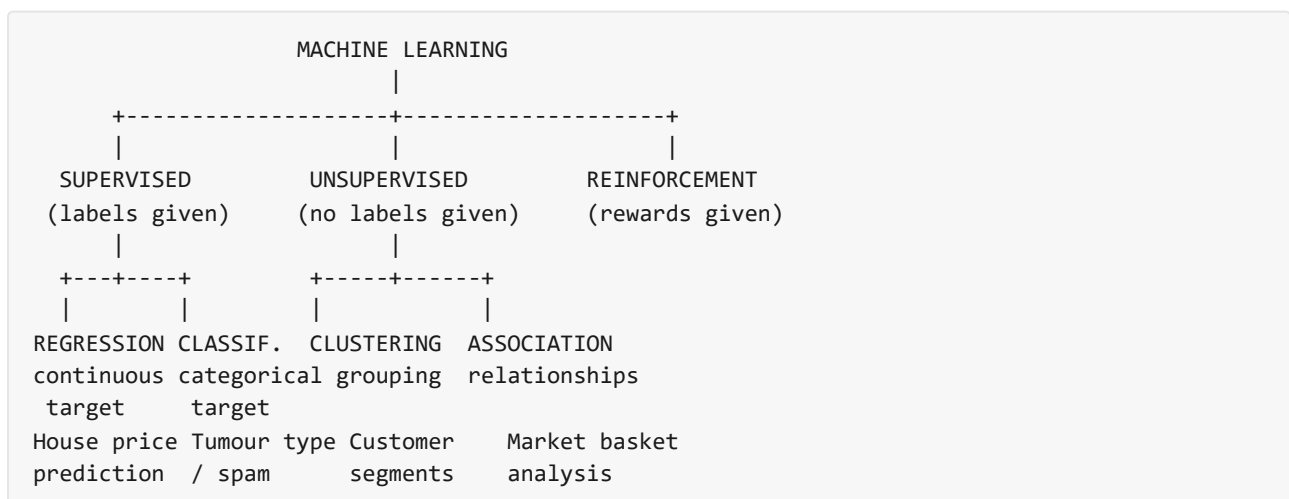
Example 6's measure is *asymmetric*: missing a fraud costs more than a false alarm. That becomes **recall versus precision** in Section 12.

Understand: why P must be measurable. **Memorize:** the definition and all six rows. **Practice:** write T, P, E for “recommend movies”.

2. Machine Learning Models

2.1 The master map

Exam priority: Medium — MCQs of the form “X is an example of which learning type?” come straight from this map.



Branch	Target	Example algorithms	Example use
Regression	Continuous	Linear Regression, Random Forest	House price
Classification	Categorical	Decision Trees, Logistic Regression, SVM	Tumour type, spam
Clustering	<i>None</i>	K-Means, Hierarchical	Customer segmentation
Association	<i>None</i>	Apriori	Market basket analysis

Understand: the split is by *what information is available*. **Memorize:** the map. **Practice:** automated vehicles (RL), fraud detection (supervised classification), customer segmentation (clustering).

2.2 Supervised and unsupervised learning

Exam priority: Medium — both definitions are direct short-answer items, and “also called exploratory learning” is flagged as an MCQ.

Supervised: training data plus desired outputs (labels). From inputs x and output Y you learn the **mapping function** $Y = f(X)$, approximating f well enough to predict Y for **unseen** x ; the approximation is the **hypothesis function**. Given $(x_1, y_1), \dots, (x_n, y_n)$: a **real-valued** y gives **regression**, a **categorical** y gives **classification**. The slides’ example trains on squares, triangles and hexagons (“four equal sides gives Square”) and then labels a new figure.

Unsupervised (exploratory learning): training data without desired outputs. Given $x_1, \dots, x_n$ with no labels, the goal is to **find the underlying structure, group** by similarity, and **represent the data compactly**. Two types:

1. **Clustering** — objects within a group are similar, objects across groups are not.
2. **Association** — **relationships between variables** in a large database; items that occur together. (*People who buy a new home are likely to buy furniture.*)

Algorithms: K-means, Hierarchical clustering, Anomaly detection, PCA, Independent Component Analysis, Apriori, SVD.

Understand: the label is the supervision; without it there is no correct answer to score against. **Memorize:** $Y = f(X)$; regression vs classification; clustering vs association. **Practice:** label a problem as one of the four branches.

2.3 Semi-supervised, self-supervised, and reinforcement learning

Exam priority: Low in this unit — the first two get one line each in the slides; RL is introduced here and examined in depth in Unit 4.

Type	Idea
Semi-supervised	A small labelled set plus a large unlabelled set . Labels are expensive (a doctor must label each scan); the few labels anchor structure found in the many unlabelled points.
Self-supervised	Labels generated from the data itself — hide a word in a sentence and predict it.

[Supplementary — not examinable] The two descriptions above are the standard textbook meanings; the slides name the terms without detail.

Reinforcement learning is **learning what to do — mapping situations to actions**. The **agent** observes a **state**, takes an **action**, and receives **feedback**: a **reward** if positive, a **punishment** if negative. It must **discover** which actions earn the most reward; it is not told.

The dog analogy (slides). Fetch the ball, get a cookie; fail, no cookie. The mapping from situation to action the dog learns is the **control policy**.

Element	Definition
Agent	The learner; acts on the environment and receives rewards
Policy function	The agent's behaviour — how it chooses an action
Value function $v(s)$	How good it is to be in state s — the total expected reward from there onward

Understand: a reward is a score, not the correct answer. **Memorize:** agent, policy, value function. **Practice:** nothing.

2.4 Comparing the paradigms; generative versus discriminative

Exam priority: Medium — the comparison table is a common short question; the generative/discriminative lists are easy marks.

Aspect	Supervised	Unsupervised	Reinforcement
Input data	Labelled	Unlabelled	An environment, no fixed dataset
Feedback	The correct answer	None	Reward or punishment
Goal	Learn $X \rightarrow Y$	Find hidden structure	Maximize total cumulative reward
Example	Spam filter	Customer segmentation	Game AI, robot navigation

	Discriminative	Generative
Models	Conditional probability $P(y x)$ — draws boundaries	The distribution of the data — how data is placed in the space
Examples	Logistic Regression, SVM, Decision Tree, Random Forest	Bayesian Network, Hidden Markov Model, GANs

Understand: RL gets a score for the action it chose, not the right action. **Memorize:** both tables.
Practice: reproduce the first table.

3. Concept Learning and Inductive Bias

3.1 Concepts, the instance space, and the concept space

Exam priority: Medium — the definitions here feed the counting numerical in 3.3, which is where the marks are.

A concept defines what belongs to a category and what does not.

The concept of the letter 'A' is whatever rule separates A's from non-A's. To make it machine-tractable, describe each object by a **feature vector** (Has Crossbar, Has Pointed Top, Symmetrical, Has Two Legs, Is Curved), so 'A' is [Yes, Yes, Yes, Yes, No].

Feature space (instance space) χ : all possible combinations of attribute values.

$$|\chi| = \prod_i |A_i| \quad (= 2^n \text{ when all } n \text{ attributes are binary})$$

Running example: a fruit with Colour (Red, Green) and Shape (Round, Long). The feature space holds $2 \times 2 = 4$ instances: RR, RL, GR, GL.

Concept space: every possible labelling of every instance — equivalently, all subsets of the instance space.

Each of the 4 instances can be labelled 1 or 0 independently, so there are $2^4 = 16$ concepts:

$$|\text{Concept space}| = 2^{|\chi|} \quad (= 2^{2^n} \text{ for } n \text{ binary attributes})$$

Why the double exponential? Each instance is a row of a truth table; a concept fills in the whole output column. With m rows there are 2^m fillings, and m is itself 2^n .

The trap: the exponent is the size of the *feature space*, the **product** of the attribute arities — it is 2^n only when every attribute is binary.

Understand: an instance is one object; a concept is a whole labelling. **Memorize:** both formulas.

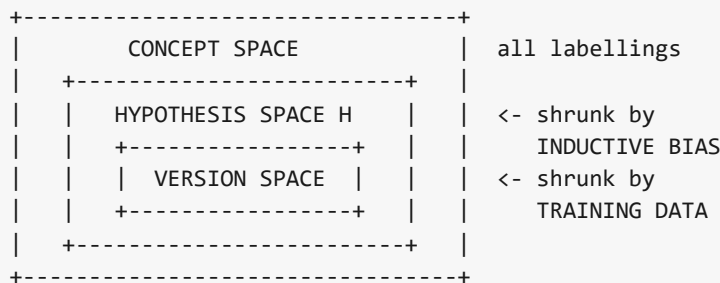
Practice: 3.3.

3.2 Inductive bias

Exam priority: Medium — the definition and diagram are asked, but usually as the lead-in to 3.3 and 3.4.

Six modest attributes already give 2^{96} concepts. No algorithm can search that, so we shrink it.

We make an assumption about the nature of the concept. That assumption reduces the space of concepts we look at. That reduction is the inductive bias. The shrunken space is the hypothesis space H .



Draw this in any concept-learning answer.

The bias we adopt: a hypothesis is a **conjunction (AND) of attribute-value constraints**. Each slot holds a specific value (the attribute **must** match), a ? (**any** value), or the empty-set symbol (**no** value is acceptable — the hypothesis rejects everything).

Understand: bias is what makes search possible; the price is that the true concept may lie outside H .

Memorize: the definition and the diagram. **Practice:** 3.4.

3.3 Counting the hypothesis space

Exam priority: High — a direct numerical; the slides work the six-attribute case end to end.

Each slot can take its attribute's values, or ?, or reject. But **one reject slot makes the whole hypothesis call everything negative**, so all such hypotheses behave identically and count as **one**:

$$|H|_{\text{semantically distinct}} = \prod_{i=1}^k (|A_i| + 1) + 1$$

The inner “+1” is ?; the final “+1” is the single **Reject-All** hypothesis.

Fruit example: $|H| = (2 + 1)(2 + 1) + 1 = 10$, down from 16 concepts: $\langle R, R \rangle, \langle R, L \rangle, \langle R, ? \rangle, \langle G, R \rangle, \langle G, L \rangle, \langle G, ? \rangle, \langle ?, R \rangle, \langle ?, L \rangle, \langle ?, ? \rangle$ (accept all), reject-all.

What the bias threw away: the six concepts that need an **OR**, such as “Red-Long OR Green-Round”. A conjunction cannot express a disjunction — which is why decision trees (Section 6.1) are more expressive.

Worked example. An object has 6 attributes: a_1 has 3 values, a_2 to a_6 are binary.

Step 1 — feature space: $|\chi| = 3 \times 2 \times 2 \times 2 \times 2 \times 2 = 3 \times 32 = 96$.

Step 2 — concept space: 2^{96} .

Step 3 — hypothesis space:

$$|H| = (3 + 1) \times (2 + 1)^5 + 1 = 4 \times 243 + 1 = 972 + 1 = 973$$

From 2^{96} to 973.

Understand: why identical-behaviour hypotheses count once. **Memorize:** $|H| = \prod (|A_i| + 1) + 1$.

Practice: the three steps with any mix of arities.

3.4 Restriction bias versus preference bias

Exam priority: High — the slides bold the ID3-versus-Find-S sentence, and it recurs in Section 7.

Type	Meaning	Example
Restriction bias	The hypothesis space is <i>restricted</i> — some hypotheses cannot be represented	Find-S, Candidate Elimination (conjunctions only)
Preference bias	Everything is representable, but the search <i>prefers</i> some hypotheses	ID3 (short trees, high-gain attributes)

The fruit bias is a **restriction bias**: some hypotheses are **never considered**, so **the target concept may or may not be in H** — it is absent if the true concept is non-conjunctive, or if the training data has errors.

The bolded sentence: the inductive bias of **ID3** follows from its search strategy; that of **Find-S** follows from the definition of its search space.

Understand: restriction limits what can be *said*; preference limits what gets *chosen*. **Memorize:** the sentence verbatim. **Practice:** name the bias type of a given algorithm.

3.5 Version space

Exam priority: Medium — a definition plus a short enumeration; the slides work one small example.

The version space is the set of all hypotheses in H consistent with the training data. A hypothesis is **consistent** if it classifies every training object correctly.

$$VS = \{h \in H : h \text{ is consistent with } D_{\text{training}}\}$$

Worked example. RR (Red, Round) is **Yes**; RL (Red, Long) is **No**; GR and GL unseen. Which hypotheses call RR positive and RL negative?

Hypothesis	RR	RL	Consistent?
$\langle R, R \rangle$	+	-	yes — in VS
$\langle R, L \rangle$	-	+	no
$\langle R, ? \rangle$	+	+	no — calls RL positive
$\langle G, R \rangle$	-	-	no — calls RR negative
$\langle G, L \rangle, \langle G, ? \rangle$	-	-	no
$\langle ?, R \rangle$	+	-	yes — in VS
$\langle ?, L \rangle$	-	+	no
$\langle ?, ? \rangle$	+	+	no
Reject All	-	-	no

$$VS = \{\langle R, R \rangle, \langle ?, R \rangle\}$$

The machine cannot yet tell whether “edible” means *red and round* or just *round*. **More data shrinks the version space**; when one hypothesis remains, learning is complete.

Understand: bias shrinks H , data shrinks VS . **Memorize:** both definitions. **Practice:** enumerate H , test each row, list VS .

4. The Find-S Algorithm

4.1 The algorithm

Exam priority: Medium — four lines to state; the marks are in the trace (4.2) and the limitations (4.3).

Find-S, the **basic concept-learning algorithm**, finds the **most specific hypothesis** consistent with all the **positive** examples. **It uses only the positive examples** — negatives are ignored. It moves from **most specific** towards **more general**, loosening the hypothesis each time it fails to cover a positive example.

```

FIND-S:
1. Initialize h to the most specific hypothesis in H
   h = <empty, empty, ..., empty>
2. For each POSITIVE training instance x:
   For each attribute constraint a_i in h:
     If a_i is satisfied by x: do nothing
     Else: replace a_i by the next more general
           constraint satisfied by x (in practice: '?')
3. Output h

```

In plain words: assume nothing is positive; on each positive example, wherever your guess disagrees with it, loosen that slot to ?.

Understand: why it starts from the most specific hypothesis. **Memorize:** the steps. **Practice:** 4.2.

4.2 Worked example — EnjoySport

Exam priority: High — this table is traced in the slides, and the question asks for the hypothesis after *every* row.

#	sky	temp	humidity	wind	water	forecast	enjoy
1	sunny	warm	normal	strong	warm	same	yes
2	sunny	warm	high	strong	warm	same	yes
3	rainy	cold	high	strong	warm	change	no
4	sunny	warm	high	strong	cool	same	yes

Step 0 — initialize: $h = \langle \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset \rangle$.

Step 1 — row 1, positive. Every slot disagrees with the empty constraint, so copy the example: $h = \langle \text{sunny, warm, normal, strong, warm, same} \rangle$.

Step 2 — row 2, positive.

```

h = { sunny, warm, normal, strong, warm, same }
x = { sunny, warm, high , strong, warm, same }
----- humidity disagrees -> ?
h = { sunny, warm,  ?  , strong, warm, same }

```

Step 3 — row 3 is negative: ignored.

Step 4 — row 4, positive.

```

h = { sunny, warm,  ?  , strong, warm, same }
x = { sunny, warm, high , strong, cool, same }
----- water disagrees -> ?
h = { sunny, warm,  ?  , strong, ?  , same }

```

Final: $h = \langle \text{sunny, warm, ?, strong, ?, same} \rangle$. A new day $\{\text{sunny, warm, high, strong, warm, same}\}$ matches every constraint (humidity and water are don't-cares), so the prediction is **yes**.

Understand: the hypothesis only ever loosens. **Memorize:** the pattern of intermediate hypotheses.

Practice: re-trace with the book shut, then Practice Q2.

4.3 Properties and limitations

Exam priority: Medium — the four limitations are a standard list question, backed by a worked counter-example in the slides.

Properties. Find-S **assumes every instance is negative** unless its knowledge shows otherwise — that is why negatives can be ignored. The EnjoySport result is consistent with negative row 3 (rainy fails the sunny constraint). **In general the output is also consistent with the negatives provided** (1) the target concept really is in H — a conjunction — **and** (2) the data has **no errors**. The answer is **not unique**: $\langle \text{sunny, ?, ?, strong, ?, same} \rangle$ also fits; Find-S returns the most specific because it is defined to. One could instead **specialize from** $\langle ?, ?, ?, ?, ?, ? \rangle$ **using negatives**; doing both is the **Candidate Elimination Algorithm**.

Syllabus exclusion. The slides state in bold: “**Candidate Elimination Algorithm (Not in syllabus!!)**”. Know that it exists and keeps a specific boundary S and a general boundary G ; do not trace it.

Limitations.

1. The most specific consistent hypothesis **may still cover a negative example** — it is not necessarily the right concept.

Slides say: the final hypothesis “may also be consistent with a negative example”. **More precisely:** the failure described is the opposite — the hypothesis becomes general enough to **cover** a negative, which makes it **inconsistent** with that example. The slides’ own counter-example below shows exactly this.

2. It is **just one of many** hypotheses that fit the training data.
3. **Noisy data misleads it:** a single mislabelled positive permanently over-generalizes h , and nothing can undo it because negatives are ignored.
4. **No backtracking** — an earlier generalization is never reconsidered.

The counter-example. a_1 has 3 values; a_2, a_3 are binary.

a_1	a_2	a_3	value
Low	1	0	YES
Low	0	0	YES
Medium	1	0	NO
High	1	1	YES

Trace: $\{\emptyset, \emptyset, \emptyset\}$, then $\{\text{Low}, 1, 0\}$, then $\{\text{Low}, ?, 0\}$, then — after the negative is skipped and $(\text{High}, 1, 1)$ forces every slot open — $\{?, ?, ?\}$, which **covers the negative (Medium, 1, 0)**. Limitation 1, demonstrated.

Understand: ignoring negatives is both the speed-up and the weakness. **Memorize:** the four limitations and the two consistency conditions. **Practice:** trace, then check the result against every negative row.

5. Bias–Variance Decomposition

5.1 Bias and variance defined

Exam priority: Medium — definitions and the dartboard picture; the marks are in 5.3 and 5.5.

Term	Definition (as the slides give it)
Bias	The difference between an estimator's expected value and the true value being estimated.
Variance	The amount by which a model's performance changes when trained on a different dataset from the same distribution .

Dartboard. High bias: darts tightly clustered but far from the bullseye — consistently wrong.
High variance: darts scattered everywhere — their average may be the bullseye, but no single throw is reliable.

Understand: bias is systematic error, variance is instability across datasets. **Memorize:** both definitions. **Practice:** nothing.

5.2 Overfitting and underfitting

Exam priority: Medium — the comparison table is asked directly; the regression and classification pictures illustrate it.

	Overfitting	Underfitting
What happens	Captures the noise along with the pattern	Fails to capture the pattern
Symptom	Good on training data, bad on test	Bad on both
Bias / Variance	Low / High	High / Low
Complexity	Too complex	Too simple

Regression picture (slides). True pattern: a curve. A straight line has some training error and a *similar* test error — **high bias, low variance**. A squiggle through every training point has **zero** training error and a large test error — **low bias, high variance**. A good model follows the curve without chasing noisy points.

Classification picture. The same three cases as boundary shapes: a line missing the true shape (underfit), a smooth curve (balanced), a wiggly boundary wrapping every point (overfit). **A model can be high bias and high variance at once** — mostly linear where the truth is quadratic, yet over-flexible in the middle; in high dimensions that is the realistic case.

Understand: both are ends of one axis, model complexity. **Memorize:** overfit = low bias, high variance; underfit = the reverse. **Practice:** name the failure from a described boundary.

5.3 Diagnosing bias and variance from error numbers

Exam priority: High — the four-column train/test table is worked in the slides and reappears as a direct question.

In high dimensions you cannot plot the data, so you diagnose from **training error versus test error**. The slides' working rule: **the training error tells you the bias; the increase from train to test tells you the variance**.

Slides say: training error is the bias and the train–test gap is the variance. **More precisely:** these are **diagnostic proxies**, not definitions. Bias is $\mathbb{E}[\hat{f}(x)] - f(x)$ and variance is the spread of $\hat{f}(x)$ across training sets (5.5) — both defined over *many* training sets, while train and test error come from *one*. The proxies hold when train and test share a distribution **and** the Bayes error is near zero. Diagnose with the proxy; define with the formula.

	Model 1	Model 2	Model 3	Model 4
Train error	1%	15%	15%	0.5%
Test error	11%	16%	30%	1%
Diagnosis	High variance	High bias	High bias and high variance	Low bias, low variance

Model 1: tiny training error, 10-point gap — variance. Model 2: 15% training error, 1-point gap — bias. Model 3: both. Model 4: the one we want.

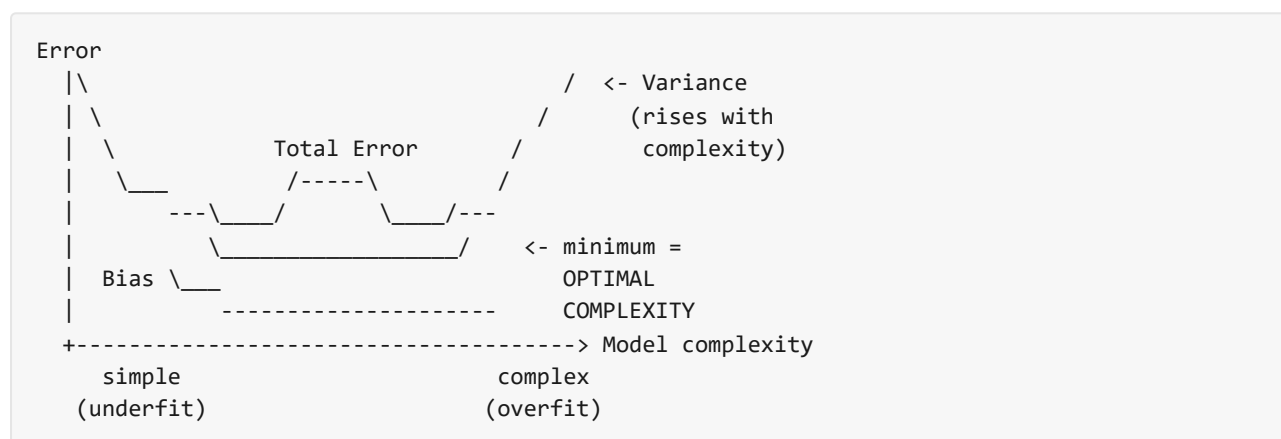
The Bayes-error caveat. This assumes the **Bayes (optimal) error** is near zero. If it were 15% — genuinely blurry images — Model 2 is **not** high bias; it is at the theoretical limit.

Compact rules. High variance: low train error, high test error. **High bias:** high train error, test error close to it.

Understand: the same numbers change meaning when the Bayes error moves. **Memorize:** the two rules and the table. **Practice:** diagnose any train/test pair with a one-line reason.

5.4 The tradeoff curve

Exam priority: Medium — a draw-and-explain question; the four statements are the marking points.

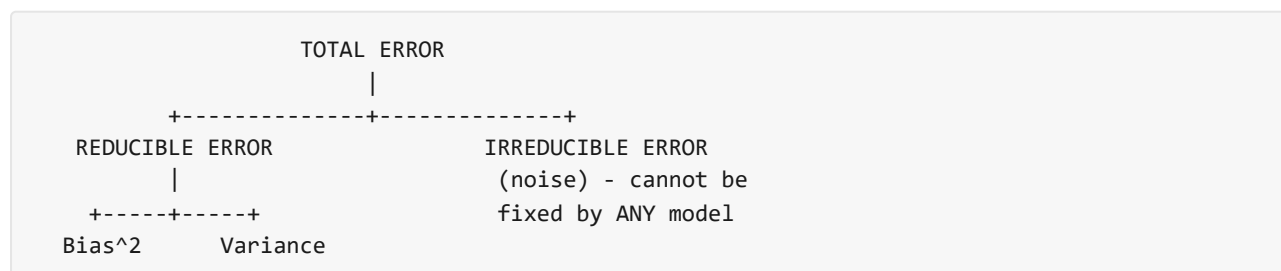


1. **Bias reduction:** as complexity rises, bias **falls** at first.
2. **Variance increase:** variance **rises**, at first more slowly than bias falls.
3. **Total error** therefore **falls** at first.
4. **Optimal complexity:** eventually variance rises sharply and total error turns upward; **the minimum is the optimal complexity.**

Understand: the minimum exists because two opposing curves are added. **Memorize:** the four statements. **Practice:** draw it with all three curves labelled.

5.5 The formal decomposition, with a worked numerical

Exam priority: High — the equation is bookwork and the three-dataset numerical is worked in full in the slides.



$$\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{(\mathbb{E}[\hat{f}(x)] - f(x))^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Irreducible}}$$

Bias: systematic error from overly simple assumptions — underfitting. **Variance:** sensitivity to which training set you got — overfitting. **Irreducible σ^2 :** inherent noise; no model reduces it. Here $\mathbb{E}$ averages **over all training sets drawn from the distribution** — which is why variance needs several training sets, and why 5.3’s single-dataset rule is only a proxy.

Worked numerical. True function $y = 2x$ for $x \in \{1, 2, 3\}$, noise $N(0, 1)$. Three training sets; the model fitted to each is a constant $y = c$. Decompose the error at $x_0 = 2$.

ACTUALS		Set 1		Set 2		Set 3	
x	y	x	y	x	y	x	y
1	2	1	1.8	1	2.1	1	2.4
2	4	2	3.9	2	3.4	2	3.3
3	6	3	5.7	3	5.6	3	6.1
		c:	3.8	c:	3.7	c:	3.93

Each c is the mean of its set: $(1.8 + 3.9 + 5.7)/3 = 11.4/3 = 3.8$; $(2.1 + 3.4 + 5.6)/3 = 11.1/3 = 3.7$; $(2.4 + 3.3 + 6.1)/3 = 11.8/3 = 3.93$. True value $f(2) = 4$; predictions 3.8, 3.7, 3.93.

Step 1 — average prediction: $(3.8 + 3.7 + 3.93)/3 = 11.43/3 = 3.81$.

Step 2 — bias: $3.81 - 4 = -0.19$, so $\text{Bias}^2 = 0.0361$.

Step 3 — variance about the mean 3.81: deviations $-0.01, -0.11, +0.12$; squares 0.0001, 0.0121, 0.0144; sum 0.0266; divide by 3: 0.00887.

Step 4 — irreducible: the noise is $N(0, 1)$, so $\sigma^2 = 1$.

Step 5 — total: $0.0361 + 0.00887 + 1 = 1.045$.

Lesson. The model’s own faults total about 0.045 — tiny beside the irreducible 1.0. No tuning pushes the error below 1.0; knowing that stops you chasing an impossible target.

Understand: $\mathbb{E}$ ranges over datasets, not data points. **Memorize:** the three-term equation. **Practice:** mean prediction, bias-squared, variance, add σ^2 — in that order.

6. Decision Tree Learning and ID3

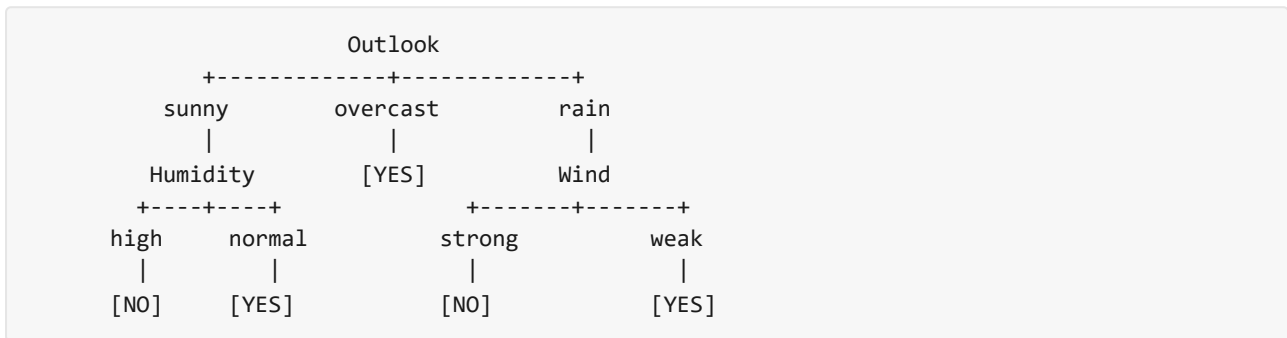
6.1 What a decision tree is, and why it is more expressive than Find-S

Exam priority: Medium — terminology is assumed rather than asked; the “disjunction of conjunctions” comparison is a recurring theory question.

A **decision tree** is a flowchart-like diagram that reaches a decision through a series of conditions; it can be **re-represented as if-then rules**, and serves **classification and regression**.

Instances are **sorted from the root to a leaf**. The **root node** is where building starts; a **decision node** tests one attribute and branches on its value; a **leaf** carries the class label. Every non-leaf is a decision node; every branch is one attribute value.

The Play Tennis tree, derived in 6.4:



Read as logic, the decision is **yes** whenever

$$(\text{Sunny} \wedge \text{Normal}) \vee (\text{Overcast}) \vee (\text{Rain} \wedge \text{Weak})$$

Decision trees represent a DISJUNCTION OF CONJUNCTIONS — more expressive than Find-S. Each root-to-leaf path is an AND; the paths are joined by OR. Section 3.3 showed Find-S cannot express the six fruit concepts that need an OR; a tree can express all of them.

Understand: one path is an AND, the set of paths is an OR. **Memorize:** the three node types and the phrase “disjunction of conjunctions”. **Practice:** convert a drawn tree into its logical expression.

6.2 Building a tree; ID3 versus CART

Exam priority: Low — a procedure summary and a comparison table; the examinable content is entropy and gain in 6.3.

1. Select the "best" attribute for the next node <- THE CRUX
2. Make it the decision attribute for the node
3. For each value, create a branch; a branch needing no further decision is a LEAF
4. Sort the training examples down to the new nodes
5. If perfectly classified, STOP; else iterate on the leaves

Step 1 is the whole problem. Different definitions of “best” give different trees for the same data; which one we *prefer* is ID3’s inductive bias (7.2).

	ID3 (Iterative Dichotomiser 3)	CART
Created by	Ross Quinlan, 1986	Leo Breiman
Tree shape	Multiway	Binary
Heuristic	Largest information gain (entropy)	Gini impurity for classification; least squares for regression
Used for	Classification	Classification and regression

Slides say: ID3 is used for “classification and regression”. **More precisely:** ID3 as defined splits on **discrete-valued targets only**, so it is a classification algorithm; entropy is undefined for a continuous target, which is exactly why CART introduced the least-squares criterion for regression.

Understand: the split criterion is the only real design choice. **Memorize:** the table. **Practice:** nothing.

6.3 Entropy, information gain, and the ID3 algorithm

Exam priority: High — every ID3 numerical starts here, and the formulas are asked on their own.

We want a decision node whose subsets are **as homogeneous as possible**.

Entropy measures non-homogeneity, uncertainty, disorder. Higher entropy means messier data; entropy needs to be reduced.

$$\text{Entropy}(S) = - \sum_{i=1}^c p_i \log_2 p_i \quad (\text{binary: } -p_+ \log_2 p_+ - p_- \log_2 p_-)$$

with c classes and p_i the fraction of S in class i . **Three buckets (slides):** all red — complete knowledge, entropy **0**; 75% red / 25% green — partial, **medium**; 50/25/25 — least knowledge, **high**. **Most knowledge means least entropy.**

Conventions. $0 \log_2 0 = 0$. With c classes entropy can reach $\log_2 c$: 1 for binary, $\log_2 3 = 1.585$ for three. A three-class entropy above 1 is normal.

Split	Entropy		Split	Entropy
pure	0		2:3	0.971
1:1	1.000		3:4	0.985
1:2	0.918		1:6	0.592
1:3	0.811		5:9	0.940

Information gain measures how effective an attribute is at classifying the data: entropy before the split minus entropy after. Higher is better. It is the **expected reduction in entropy** from partitioning on the attribute.

$$I(A) = \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \text{Entropy}(S_v) \quad \text{Gain}(S, A) = \text{Entropy}(S) - I(A)$$

The weight $|S_v|/|S|$ is the probability of landing in that branch — a subset of 10 of 14 examples matters more than one of 1.

ID3(S):

1. Compute Entropy(S)
2. For each attribute A: compute Entropy(S_v) for each value v, then I(A), then Gain(S,A) = Entropy(S) - I(A)
3. Pick the attribute with the HIGHEST GAIN as the decision node
4. Split on it; a branch with Entropy(S_v) = 0 becomes a LEAF, any other branch recurses with the remaining attributes
5. Repeat until every branch terminates

Understand: entropy is disorder; gain is the expected drop in disorder. **Memorize:** both formulas, the steps, the quick values. **Practice:** 6.4.

6.4 Worked example — the Play Tennis trace

Exam priority: High — the largest numerical in the unit; worked end to end in the slides and repeated in the lab.

#	Outlook	Temp	Humidity	Windy	Play
1	Sunny	High	High	Weak	No
2	Sunny	High	High	Strong	No
3	Overcast	High	High	Weak	Yes
4	Rainy	Medium	High	Weak	Yes
5	Rainy	Cool	Normal	Weak	Yes
6	Rainy	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Medium	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rainy	Medium	Normal	Weak	Yes
11	Sunny	Medium	Normal	Strong	Yes
12	Overcast	Medium	High	Strong	Yes
13	Overcast	High	Normal	Weak	Yes
14	Rainy	Medium	High	Strong	No

Level 0 – whole set. $|S| = 14$, 9 Yes, 5 No:

$$\text{Entropy}(S) = -\left(\frac{9}{14} \log_2 \frac{9}{14} + \frac{5}{14} \log_2 \frac{5}{14}\right) = 0.409 + 0.531 = 0.940$$

Outlook.

Subset	Rows	Yes : No	Entropy
Sunny	1,2,8,9,11	2 : 3	0.971
Overcast	3,7,12,13	4 : 0	0
Rain	4,5,6,10,14	3 : 2	0.971

$$I = \frac{5}{14}(0.971) + \frac{4}{14}(0) + \frac{5}{14}(0.971) = 0.347 + 0 + 0.347 = 0.693 \quad \text{Gain} = 0.940 - 0.693 = \mathbf{0.247}$$

Temp.

Subset	Rows	Yes : No	Entropy
High	1,2,3,13	2 : 2	1
Medium	4,8,10,11,12,14	4 : 2	0.918
Cool	5,6,7,9	3 : 1	0.811

$$I = \frac{4}{14}(1) + \frac{6}{14}(0.918) + \frac{4}{14}(0.811) = 0.286 + 0.393 + 0.232 = 0.911 \quad \text{Gain} = \mathbf{0.029}$$

Humidity.

Subset	Rows	Yes : No	Entropy
High	1,2,3,4,8,12,14	3 : 4	0.985
Normal	5,6,7,9,10,11,13	6 : 1	0.592

$$I = \frac{7}{14}(0.985) + \frac{7}{14}(0.592) = 0.4925 + 0.296 = 0.7885 \quad \text{Gain} = 0.940 - 0.7885 = \mathbf{0.152}$$

Windy.

Subset	Rows	Yes : No	Entropy
Weak	1,3,4,5,8,9,10,13	6 : 2	0.811
Strong	2,6,7,11,12,14	3 : 3	1

$$I = \frac{8}{14}(0.811) + \frac{6}{14}(1) = 0.463 + 0.429 = 0.892 \quad \text{Gain} = 0.940 - 0.892 = \mathbf{0.048}$$

Decision: Outlook 0.247 beats Humidity 0.152, Windy 0.048, Temp 0.029. **Root = Outlook.**
Overcast has entropy 0 and becomes a **leaf YES** at once.

Level 1 — Sunny branch. Rows 1, 2, 8, 9, 11; entropy 0.971.

#	Temp	Humidity	Windy	Play
1	High	High	Weak	No
2	High	High	Strong	No
8	Medium	High	Weak	No
9	Cool	Normal	Weak	Yes
11	Medium	Normal	Strong	Yes

Temp: High = 1,2 (both No, 0); Medium = 8,11 (1:1, entropy 1); Cool = 9 (0). $I = \frac{2}{5}(1) = 0.4$; **Gain = 0.571.** **Humidity:** High = 1,2,8 (all No, 0); Normal = 9,11 (all Yes, 0). $I = 0$; **Gain = 0.971.** **Windy:** Weak = 1,8,9 (1:2, 0.918); Strong = 2,11 (1:1, 1). $I = \frac{3}{5}(0.918) + \frac{2}{5}(1) = 0.951$; **Gain = 0.020.**

Humidity wins with the maximum possible gain; both branches have entropy 0 and are leaves.

Level 1 — Rain branch. Rows 4, 5, 6, 10, 14; entropy 0.971.

#	Temp	Humidity	Windy	Play
4	Medium	High	Weak	Yes
5	Cool	Normal	Weak	Yes
6	Cool	Normal	Strong	No
10	Medium	Normal	Weak	Yes
14	Medium	High	Strong	No

Windy: Weak = 4,5,10 (all Yes, 0); Strong = 6,14 (all No, 0). $I = 0$; **Gain** = **0.971**, the maximum, so **Windy** is chosen and both branches are pure leaves. The final tree is the one drawn in 6.1.

Notice: Temperature never appears. Once Outlook, Humidity and Wind are known, it carries no further information.

Understand: recursion stops exactly when a subset's entropy is 0. **Memorize:** the order — Entropy(S), subset entropies, $I(A)$, Gain, pick the max. **Practice:** reproduce this trace from the raw table.

6.5 A second trace, and a multi-class target

Exam priority: Medium — smaller variants of 6.4; the value is the slide-error note and the multi-class ceiling.

Job Acceptance.

salary	Location	accepted
Tier 1	MUM	YES
Tier 1	BLR	YES
Tier 2	BLR	NO
Tier 1	HYD	NO
Tier 2	MUM	YES

Slides say: the table has 6 rows. **More precisely:** the sixth row duplicates (Tier 1, HYD, NO), and the slide's own solution uses $|S| = 5$ — it even warns “ensure there are no duplicate rows”.

$|S| = 5$, 3 Yes, 2 No: Entropy(S) = 0.971.

salary: Tier 1 = 3 rows at 2:1 (0.918); Tier 2 = 2 rows at 1:1 (1). $I = \frac{3}{5}(0.918) + \frac{2}{5}(1) = 0.5508 + 0.4 = 0.9508$; **Gain** = **0.0202**. **Location:** MUM = 2 rows, both Yes (0); BLR = 2 rows at 1:1 (1); HYD = 1 row (0). $I = \frac{2}{5}(0) + \frac{2}{5}(1) + \frac{1}{5}(0) = 0.4$; **Gain** = **0.571**.

Root = Location. MUM and HYD are pure leaves; BLR still mixes one Yes and one No, and the remaining attribute salary separates them (Tier 1 YES, Tier 2 NO).

Three-class target. For 8 rows with target values T (3), E (2), F (3):

$$\text{Entropy}(S) = -\left(\frac{3}{8} \log_2 \frac{3}{8} + \frac{2}{8} \log_2 \frac{2}{8} + \frac{3}{8} \log_2 \frac{3}{8}\right) = 0.531 + 0.5 + 0.531 = 1.561$$

The ceiling is $\log_2 3 = 1.585$, so this is near-maximum disorder. **Binary entropy never exceeds 1; three-class entropy can.**

Understand: the procedure does not change with the number of classes. **Memorize:** the $\log_2 c$ ceiling. **Practice:** Practice Q3.

7. ID3: Hypothesis Space, Search, and Inductive Bias

7.1 ID3's hypothesis space and search

Exam priority: Medium — the Find-S contrast table is the usual form of the question.

The hypothesis space of ID3 is the set of all possible decision trees. ID3 searches it **simple-to-complex**, with **gain as the heuristic** and a **greedy strategy**, and **outputs a single tree** that is **not guaranteed to be optimal**. Find-S's space was *restricted*; ID3's is **complete** — every tree is representable, so the target is always there, and the risk is that the *search* misses it.

- **No backtracking:** ID3 accepts the first tree it generates; consistent alternatives are never explored. The search is **incomplete**.
- **Statistically based choices:** at each node ID3 uses **all the instances** in that branch, whereas Find-S uses one example at a time. So ID3 is **robust to noise** — one bad example cannot dominate a statistic over many — and missing values can be replaced by the mode at that node.

	Find-S	ID3
Hypothesis space	Restricted to conjunctions	Complete — all trees
Expressiveness	Conjunction	Disjunction of conjunctions
Uses negatives?	No	Yes
Examples considered	One at a time	All instances in the branch
Noise robustness	Poor	Robust
Bias arises from	Definition of the search space	Search strategy
Backtracking	No	No

Understand: complete space with incomplete search, against incomplete space. **Memorize:** the table. **Practice:** reproduce it.

7.2 ID3's inductive bias and Occam's razor

Exam priority: High — the slides bold the two-part statement, and it pairs with 3.4 in most theory papers.

ID3 **prefers shortest trees (Occam's razor) and prefers high-gain attributes nearest the root** — a **preference bias** from its search strategy. It should heuristically find the shortest consistent tree. The bias also **favours attributes with many values**, since those are more likely to reduce entropy and increase gain.

Occam's razor: *prefer the simplest hypothesis that fits*. There are far fewer short trees, so a short tree that fits is less likely to be a coincidence.

The many-values problem. Add a "Date" attribute to Play Tennis. Every row has a unique date, so Date gives 14 pure subsets, $I(\text{Date}) = 0$ and $\text{Gain} = 0.94$, the maximum. ID3 would root the tree at Date, memorize the training set and generalize to nothing — **the many-values bias is a direct route to overfitting**.

	Preference (search) bias	Restriction (language) bias
Algorithm	ID3	Find-S, Candidate Elimination
Source	How the space is <i>searched</i>	How the space is <i>defined</i>
Space	Complete	Incomplete
Risk	May not <i>find</i> the best hypothesis	The best may not <i>exist</i> in H

Preference bias is the safer kind: the target is at least representable, so the failure is one of search, not of language.

Understand: why "shortest tree" is a preference, not a restriction. **Memorize:** the bolded statement and the strategy-versus-space line. **Practice:** explain the Date failure in three sentences.

8. Issues in Decision Tree Learning

8.1 The issues, and overfitting

Exam priority: High — the five-item list is stated directly, and the formal overfitting definition is asked verbatim.

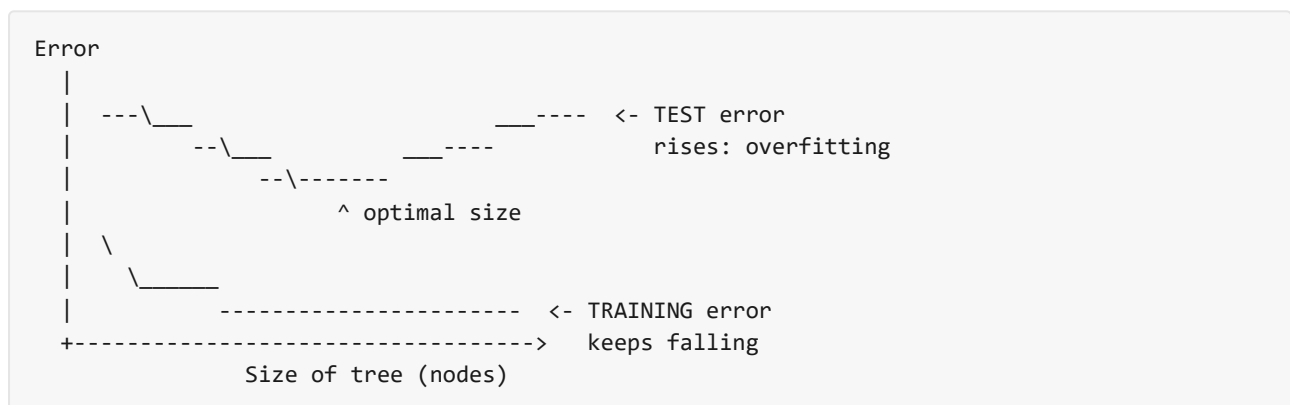
The issues: **a.** overfitting — the tree becomes too specific to the given instances; **b.** continuous attributes (Section 9); **c.** missing values; **d.** attributes with differing costs; **e.** no guarantee of a small execution time — with many choices at every step it can take enormously long.

Overfitting means becoming too specific to the training set. Causes: **noise**, a **training set that is too small**, or one that is **biased** and unrepresentative. It can occur with almost any model.

The slide's example. New instance Sunny, High, Normal, Strong, true label **No**. Sunny leads to Humidity, Normal outputs **Yes** — an error, because after Sunny the tree checks *only* Humidity; one Sunny-Normal-No training row would have forced another node. **At what depth should we stop?** The slides: **not possible to know in advance — experiment and see.**

Formal definition (verbatim). Given a hypothesis space H , $h \in H$ **overfits** the training data if there exists some alternative $h' \in H$ such that h has **smaller error than h' over the training examples**, but h' has **smaller error than h over the entire distribution of instances**.

h' comes from **a different algorithm** or from **modifying the tree ID3 built**.



Training error falls monotonically; test error falls, bottoms out, then **rises**. The gap after the minimum is overfitting.

Understand: the definition compares two hypotheses on two error measures. **Memorize:** the five issues, the definition, the three causes. **Practice:** sketch the two-curve graph.

8.2 Avoiding overfitting — pruning families and the three-way split

Exam priority: Medium — “which is better, pre- or post-pruning, and why” is a standard short question.

Either use **another algorithm**, or **modify ID3's tree** — with no guarantee it helps; experiment and see. The modification is **pruning**: cut the tree at a node, make it a leaf labelled with the **majority class** beneath it. That makes the tree **more general**.

	Early termination (pre-pruning)	Post-pruning
Idea	Stop growing before the tree fits the training data perfectly	Let it overfit , then cut back
Mechanism	Halt when a split's goodness falls below a threshold — minimum gain, maximum depth	Fully grow , then replace nodes by their majority class
Examples	Minimum gain; maximum depth	Reduced error pruning; rule post-pruning
Verdict	—	More effective , because the stopping point is hard to know in advance

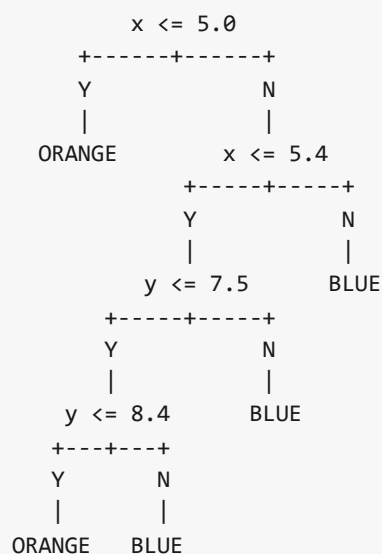
Three-way split: **training** examples build the tree; **validation** examples prune it; **test** examples give an **unbiased accuracy estimate**. Pruning on the test set would *use* it for decisions and spoil the estimate; the validation set absorbs those decisions.

Understand: post-pruning wins because the stopping point is unknowable. **Memorize:** the table and the three subsets. **Practice:** state the split and its reason.

8.3 Reduced error pruning

Exam priority: Medium — traced in the slides on a small tree; the procedure and stop rule are the marking points.

Treat **every decision node** as a pruning candidate. Pruning a node means (1) **remove its subtree**, (2) **make it a leaf**, (3) **label it with the most common class** of the training examples there. Remove a node **only if the pruned tree does no worse on the validation set**; prune **iteratively**, always removing the node whose removal **most improves** validation accuracy; **stop when further pruning hurts**.



Stage	Action	Validation error	Decision
0	Full tree	baseline	—
1	Prune the deepest node, $y \leq 8.4$, to its majority leaf	falls	keep, continue
2	Prune $y \leq 7.5$	falls	keep, continue
3	Prune the next node	rises	revert and stop

The final tree keeps $x \leq 5.0$ and $x \leq 5.4$ with leaves ORANGE, ORANGE, BLUE.

Quote: “When pruning begins the tree is at its maximum size and lowest validation accuracy. As pruning proceeds, the number of nodes falls and validation accuracy rises.”

Understand: the validation set referees every step. **Memorize:** the three steps and the stop rule.

Practice: prune, measure, keep-if-not-worse, repeat, stop-when-harmful.

8.4 Rule post-pruning

Exam priority: Medium — usually asked as “compare the two post-pruning methods”.

1. **Convert the tree to rules** — each root-to-leaf path becomes if $A_1 \wedge A_2 \wedge \dots$ then C , the A_i being **preconditions (antecedents)** and C the **postcondition (consequent)**; e.g. if not-windy and hot and sunny then bad.
2. **Prune each rule** by dropping any precondition whose removal **does not worsen estimated accuracy** on the validation set.
3. **Sort the rules by estimated accuracy** and apply them **in that order**.

Why it is more flexible: different pruning decisions can be made for different rules.

Removing a tree node affects *every* path through it; a rule can drop a condition that another rule keeps.

Understand: rules decouple paths that a tree forces to share nodes. **Memorize:** the three steps and the flexibility argument. **Practice:** write the Play Tennis tree as rules.

8.5 Tree size, missing values, and attribute costs

Exam priority: Low — three short slide lists; know the names and one-line strategies.

Correct tree size. Stop early or prune afterwards (post-pruning is more effective); use a **separate validation set**; apply a **statistical test** to judge whether expanding or pruning a node helps beyond the training set; or the **Minimum Description Length (MDL) principle**, combining depth and cost into one measure.

Syllabus note. The slides mark MDL “out of scope of the syllabus to discuss”. Know the name only.

Missing values (blood tests exist for only some patients). Strategy 1: the value **most common at that node**. Strategy 2: the value most common at that node **among examples with the same classification** — usually better.

Differing costs. Temperature is nearly free; BiopsyResult is expensive and invasive. Prefer **low-cost attributes where possible**, high-cost ones **only when needed**, by **adding a cost term to the selection measure** — e.g. maximize $\text{Gain}^2/\text{Cost}$.

Understand: each is a small change to the criterion or the data. **Memorize:** the two missing-value strategies and the cost idea. **Practice:** nothing.

9. Continuous-Valued Attributes and Decision Boundaries

9.1 Threshold selection for a continuous attribute

Exam priority: High — the 40/48/60/72/80/90 example is worked in the slides and the procedure is directly examinable.

Plain ID3 handles **discrete attributes only**. The fix: **dynamically define a Boolean attribute A_c that is TRUE if $A < c$ and FALSE otherwise**. The only question is the threshold c .

Choose the c with the **greatest information gain**. **Sort by A ; find adjacent examples that DIFFER in class; take the midpoints between them as candidates**; evaluate each by gain.

1. SORT the examples by A
2. Wherever the CLASS LABEL CHANGES between neighbours,
candidate c = midpoint of the two A values
3. For each c , form $(A < c)$ and compute $\text{Gain}(S, A_c)$
4. Choose the c with MAXIMUM gain
5. Use $(A < c)$ as an ordinary Boolean attribute in ID3

Why only class-change boundaries? A threshold between two same-class examples splits a homogeneous run and cannot improve purity. Checking only the boundaries is correct and far cheaper.

Worked example.

Temp	40	48	60	72	80	90
Play	no	no	yes	yes	yes	no

Already sorted. The class changes at 48/60 and 80/90:

$$c_1 = \frac{48 + 60}{2} = 54 \quad c_2 = \frac{80 + 90}{2} = 85$$

$|S| = 6$, 3 yes, 3 no: $\text{Entropy}(S) = 1$.

$c_1 = 54$: left {40, 48} is 0:2, entropy 0; right {60, 72, 80, 90} is 3:1, entropy 0.811.

$$I = \frac{2}{6}(0) + \frac{4}{6}(0.811) = 0.541 \quad \text{Gain} = 1 - 0.541 = \mathbf{0.459}$$

$c_2 = 85$: left five values are 3:2, entropy 0.971; right {90} is pure.

$$I = \frac{5}{6}(0.971) + \frac{1}{6}(0) = 0.809 \quad \text{Gain} = 1 - 0.809 = \mathbf{0.191}$$

Threshold 54 wins and the tree tests whether Temp is below 54.

Slides say: A_c is “ $A < c$ ”, then the worked slide names the winner “Temperature > 54”. **More precisely:** these are the same split with the branches swapped; the direction is a convention and the gain is identical.

Understand: why the candidate set is tiny. **Memorize:** the five steps and “ A_c true if $A < c$ ”.

Practice: sort, find changes, midpoints, gains, pick the best.

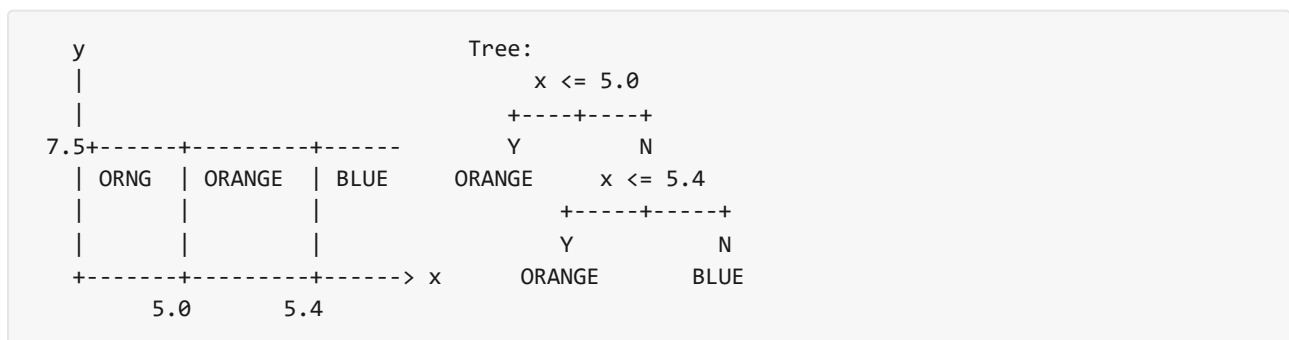
9.2 The decision boundary of a decision tree

Exam priority: Medium — named in the syllabus as “decision boundary (x–y axis)”; the slides show it through the pruning example.

With continuous features, **each node tests one feature against one threshold**, which is a line **perpendicular to that feature’s axis**.

A decision tree’s boundary is always AXIS-PARALLEL: horizontal and vertical cuts carving the plane into RECTANGLES (hyper-rectangles in higher dimensions), one class per rectangle.

The pruned tree from 8.3 on the plane:



Each $x \leq c$ is a **vertical** line, each $y \leq c$ a **horizontal** line; the leaves are the tiles.

Property	Consequence
Axis-parallel	A diagonal true boundary becomes a staircase
Deeper tree	More, smaller rectangles; jagged; overfitting
Shallower tree	Few large rectangles; underfitting
Piecewise constant	One prediction everywhere inside a rectangle

The three Unit 1 classifiers are compared by boundary shape in 11.6.

Understand: one test is one axis-parallel cut. **Memorize:** “axis-parallel rectangles” and the depth–overfitting link. **Practice:** draw the tiles for any two-feature tree.

10. Logistic Regression

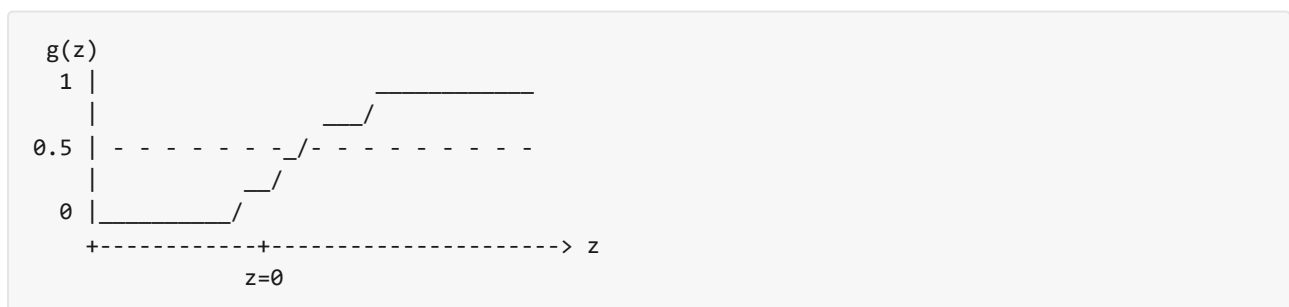
10.1 Why not linear regression, and the sigmoid hypothesis

Exam priority: Medium — the tilting-line argument and the sigmoid formula are the slides’ opening and appear as short questions.

Binary classification: $y \in \{0, 1\}$, 0 the negative class. **Attempt 1:** fit a line to tumour-size versus malignancy and predict 1 when $h_\theta(x) \geq 0.5$. **Why it fails:** one more malignant tumour far to the right **tilts the line**, and previously correct points cross the 0.5 mark. Worse, a line outputs values **above 1 and below 0**. So we want an output in $[0, 1]$ — **that is logistic regression, a classification algorithm** despite the name (it regresses onto a *probability*).

Apply the **sigmoid (logistic) function** to the linear output:

$$h_\theta(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}} \quad g(z) = \frac{1}{1 + e^{-z}}$$



$g(z) \rightarrow 0$ as $z \rightarrow -\infty$, $g(0) = 0.5$, $g(z) \rightarrow 1$ as $z \rightarrow +\infty$ — a smooth S-shaped squash.

Interpretation: $h_\theta(x)$ is the **estimated probability that $y = 1$** :

$$h_\theta(x) = P(y = 1 \mid x; \theta), \quad P(y = 1 \mid x; \theta) + P(y = 0 \mid x; \theta) = 1$$

Slide quiz: if $h_\theta(x) = 0.2$, the estimate of $P(y = 1 \mid x; \theta)$ is **0.2** and of $P(y = 0 \mid x; \theta)$ is **0.8**.

Understand: the line has no reason to stay in $[0, 1]$; the sigmoid forces it. **Memorize:** the formula, the three landmark values, the probability reading. **Practice:** the 0.2 quiz.

10.2 The decision boundary

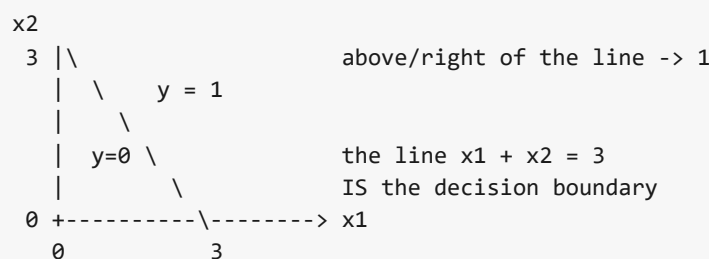
Exam priority: High — both the linear and the circular case are worked in the slides; “derive the boundary from θ ” is a standard numerical.

Since $g(z) \geq 0.5$ exactly when $z \geq 0$:

$$\text{Predict } y = 1 \iff \theta^T x \geq 0 \quad \text{Predict } y = 0 \iff \theta^T x < 0$$

The boundary is $\theta^T x = 0$ — a property of the *parameters*, not the data.

Linear. $h_\theta(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$ with $\theta = [-3, 1, 1]$. Predict 1 when $-3 + x_1 + x_2 \geq 0$, i.e. $x_1 + x_2 \geq 3$. **The boundary is the line $x_1 + x_2 = 3$** ; above-right is 1, below-left is 0.



Non-linear. Add higher-order terms: $h_\theta(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2)$ with $\theta = [-1, 0, 0, 1, 1]$. Predict 1 when $-1 + x_1^2 + x_2^2 \geq 0$, i.e. $x_1^2 + x_2^2 \geq 1$: **the unit circle**, inside 0, outside 1. More polynomial terms give more complex boundaries — and more variance (Section 5).

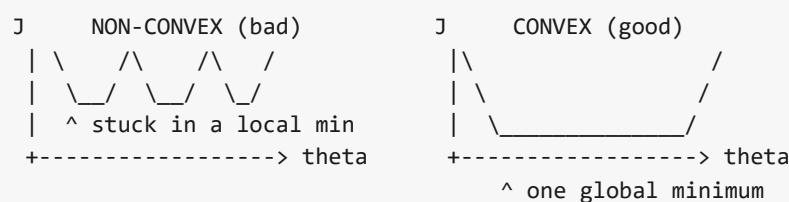
Understand: the boundary is where the sigmoid input is zero. **Memorize:** $\theta^T x = 0$. **Practice:** both cases, then Practice Q6.

10.3 The cost function

Exam priority: High — the non-convexity argument, the $-\log$ intuition and the merged formula are all asked.

Setup: m examples, n features with $x_0 = 1$, so θ has $n + 1$ entries.

Why not squared error? Linear regression’s $J(\theta) = \frac{1}{m} \sum \frac{1}{2} (h_\theta(x^{(i)}) - y^{(i)})^2$ becomes **non-convex** once the sigmoid is inside — many local minima, so gradient descent may miss the global one.



$$\text{Cost}(h_\theta(x), y) = \begin{cases} -\log(h_\theta(x)) & \text{if } y = 1 \\ -\log(1 - h_\theta(x)) & \text{if } y = 0 \end{cases}$$

Why $-\log$? If $y = 1$ and the model says $h_\theta(x) = 0$ — a malignant tumour declared benign with full certainty — the cost is **infinite**. For $y = 1$: predict 1, cost 0; predict 0.5, cost 0.693; predict 0, cost ∞ .

Merged form, so gradient descent has one expression:

$$\text{Cost}(h_\theta(x), y) = -y \log(h_\theta(x)) - (1 - y) \log(1 - h_\theta(x))$$

Verify: $y = 1$ kills the second term, leaving $-\log h_\theta(x)$; $y = 0$ kills the first, leaving $-\log(1 - h_\theta(x))$. Since y takes only those values, the forms are equivalent.

Over all m examples (log loss, cross-entropy):

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_\theta(x^{(i)})) \right]$$

Fit by minimizing $J(\theta)$; predict by outputting $h_\theta(x)$ with the fitted θ .

Understand: convexity is what makes gradient descent safe. **Memorize:** both forms and the verification. **Practice:** show the merged form reduces to each case.

10.4 Gradient descent

Exam priority: Medium — the update rule and the three remarks are quoted from the slides; usually a follow-on to 10.3.

Start from a random θ ; compute the **slope** of the cost; step **downhill**; repeat. (*A blindfolded hiker feeling for the steepest descent.*)

$$\theta_j := \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j} \quad \implies \quad \theta_j := \theta_j - \frac{\alpha}{m} \sum_{i=1}^m (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

Slides say: $\theta_j := \theta_j - \alpha \sum_i (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)}$, with no $1/m$. **More precisely:** the derivative of $J(\theta)$ above carries the $1/m$. Dropping it only rescales α , so it is the same algorithm — but keep the $1/m$ if asked to *derive* the update.

α is the **learning rate**: too big overshoots the minimum, too small converges very slowly.

1. **The rule looks identical to linear regression's.** Only the hypothesis differs — $\theta^T x$ against $1/(1 + e^{-\theta^T x})$ — so the computations differ. **Do not say they are the same algorithm.**
2. **Update all θ_j simultaneously:** compute every new value first, then assign. A vectorized implementation beats a loop.
3. **Feature scaling helps:** features on very different scales slow gradient descent.

Understand: the gradient points uphill, so subtract it. **Memorize:** the rule and the three remarks.

Practice: explain remark 1 in two sentences.

10.5 Multi-class: one-vs-all

Exam priority: Low — a short procedure given one slide.

For $y \in \{1, 2, 3, \dots\}$ (email tagging; not-ill / flu / cold): **train one classifier** $h_{\theta}^{(i)}(x)$ **per class** i , each predicting $P(y = i)$ against everything else, then **predict the class with the highest probability**:

$$\text{prediction} = \arg \max_i h_{\theta}^{(i)}(x)$$

```
3 classes -> 3 binary classifiers:
h1: class 1 vs (2 + 3)    h2: class 2 vs (1 + 3)
h3: class 3 vs (1 + 2)    predict argmax_i h_i(x)
```

Understand: k classes need k binary classifiers. **Memorize:** the argmax rule. **Practice:** nothing.

11. Instance-Based Learning — k-Nearest Neighbours

11.1 Eager versus lazy learning, and the k-NN idea

Exam priority: Medium — the comparison table is a regular short question and the inductive-bias sentence is quoted verbatim in the slides.

	Eager learning	Lazy learning
Core idea	Builds a model once and uses it for all queries	Processes the training data afresh for every query
Global model?	Yes	None
At query time	The model predicts; training data is not touched	Training data is used directly
Training / prediction cost	High / low	Zero / high
Example	Decision tree	k-NN

Instance-based (memory-based) learning compares new instances with training instances **stored in memory**. Computation is **postponed until a query arrives**, hence **lazy**. Examples: k-NN, kernel machines, RBF networks.

k-NN is a lazy algorithm that **classifies by its neighbours**, serves **classification and regression**, and rests on the idea that **similar things stay together**.

Inductive bias of k-NN: the classification of an instance x will be MOST SIMILAR to the classification of the k other instances that are NEARBY.

All instances are points in n-D space	Given a query, -> locate it in the space	Find its k nearest neighbours	Predict the MODE -> (classification) or MEAN (regression)
---	--	-------------------------------------	---

Understand: lazy methods trade training time for prediction time. **Memorize:** the table and the bias sentence. **Practice:** name one eager and one lazy algorithm with a reason.

11.2 Distance measures and the prediction rules

Exam priority: Medium — Euclidean and Manhattan are asked by name; every numerical in 11.3–11.7 depends on them.

Minkowski distance over d attributes:

$$d(x, y) = \left(\sum_{i=1}^d |x_i - y_i|^q \right)^{1/q}$$

$q = 1$ gives **Manhattan**, $\sum |x_i - y_i|$ — block-walking distance in a grid city. $q = 2$ gives **Euclidean**, $\sqrt{\sum (x_i - y_i)^2}$.

Classification, class set V : the **mode** of the k neighbours' labels.

$$\hat{f}(x_q) = \arg \max_{v \in V} \sum_{i=1}^k \delta(v, f(x_i))$$

Regression: the **mean** of the k neighbours' values.

$$\hat{f}(x_q) = \frac{1}{k} \sum_{i=1}^k f(x_i)$$

Understand: q only changes how differences combine. **Memorize:** Minkowski with $q = 1, 2$; mode versus mean. **Practice:** both distances between two 4-feature points.

11.3 Worked example — k-NN classification

Exam priority: High — this four-feature medical problem is worked in the slides and the lab repeats it; distance table, sort, mode is the standard numerical.

Assess the risk for a patient with BP 100, sugar 135, haemoglobin 12, WBC 8. $k = 3$, Euclidean.

#	BP	Sugar	Hb	WBC	Risk
A	100	120	12	6	No
B	110	130	14	5	Yes
C	120	110	11	7	Yes
D	100	140	13	7	No
E	115	140	11	6	Yes

Query $x_q = (100, 135, 12, 8)$.

$$d(x_q, A) = \sqrt{0^2 + 15^2 + 0^2 + 2^2} = \sqrt{229} = 15.13$$

$$d(x_q, B) = \sqrt{10^2 + 5^2 + 2^2 + 3^2} = \sqrt{100 + 25 + 4 + 9} = \sqrt{138} = 11.75$$

$$d(x_q, C) = \sqrt{20^2 + 25^2 + 1^2 + 1^2} = \sqrt{400 + 625 + 1 + 1} = \sqrt{1027} = 32.05$$

$$d(x_q, D) = \sqrt{0^2 + 5^2 + 1^2 + 1^2} = \sqrt{27} = 5.20$$

$$d(x_q, E) = \sqrt{15^2 + 5^2 + 1^2 + 2^2} = \sqrt{225 + 25 + 1 + 4} = \sqrt{255} = 15.97$$

Sorted: **D 5.20 (No), B 11.75 (Yes), A 15.13 (No)**, then E 15.97, C 32.05. Mode of the three: **No — the patient is assessed as not at risk.**

Understand: the answer can flip with k — at $k = 5$ the vote is 3 Yes to 2 No. **Memorize:** distance, sort, mode. **Practice:** redo with Manhattan (Practice Q7).

11.4 Worked example — k-NN regression and the error calculation

Exam priority: Medium — the slides work a weight-prediction example and a small MSE; regression is asked less often than classification.

Predict the weight of ID 11, $k = 3$, Euclidean. The slides give the distances.

Sorted: **id 6 at 2.022 (weight 60), id 5 at 2.118 (72), id 4 at 4.019 (59)**, then id 10 at 6.000, id 1 at 7.017.

$$\hat{f}(x_{11}) = \frac{60 + 72 + 59}{3} = \frac{191}{3} = 63.67$$

Scoring on a test set (slides' procedure): split 80:20, fix the distance and k , predict each test point, then compute the **error rate** (FP + FN)/all for classification or the **MSE** for regression.

MSE example. Training points with values: (3,5) 3.5, (3,6) 2.6, (4,6) 2.8, (4,4) 3.1, (7,10) 8.7, (8,9) 8.5, (8,8) 9.3, (9,10) 9.7. Test: (5,5) true 3.7, (7,8) true 8.4; $k = 4$.

For (5,5) the four nearest are (4,6), (4,4), (3,5), (3,6):

$$\hat{y}_1 = \frac{2.8 + 3.1 + 3.5 + 2.6}{4} = \frac{12.0}{4} = 3.0 \quad e_1 = (3.7 - 3.0)^2 = 0.49$$

For (7,8) they are (8,8), (8,9), (7,10), (9,10):

$$\hat{y}_2 = \frac{9.3 + 8.5 + 8.7 + 9.7}{4} = \frac{36.2}{4} = 9.05 \quad e_2 = (8.4 - 9.05)^2 = 0.4225$$

$$\text{MSE} = \frac{0.49 + 0.4225}{2} = 0.456$$

Error-rate example. Label the four points near (3,5) red and the four near (8,9) green; test (5,5) is red, (4,7) is green; $k = 4$. Both test points' four nearest neighbours are red, so one prediction is wrong: **error** = $1/2 = 0.5$ — the green point sits beside the red cluster, and eight training points cannot outvote that.

Understand: only mode-versus-mean changes. **Memorize:** MSE. **Practice:** the MSE calculation.

11.5 Choosing k

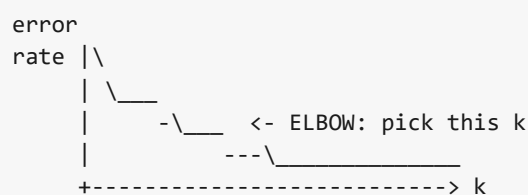
Exam priority: Medium — the slides run a small experiment and derive rules of thumb; one rule needs a correction.

Points A(2,8) triangle, B(3,9) triangle, C(5,11) circle, D(6,13) circle; query (4,10). Distances: A 2.83, B 1.414, C 1.414, D 3.61.

k	Neighbours	Outcome
1	B or C — tied at 1.414	cannot choose a neighbour
2	B triangle, C circle	1–1 vote
3	B, C, A	triangle
4	B, C, A, D	2–2 vote

Rules from the slides: there is **no right k to start with**; try $k = (\text{classes}) + 1$ and **on a tie decrease k by 1**; $k = \sqrt{n}$ also performs well; the **elbow method** — plot error rate against k and pick the elbow, redoing it whenever the data changes.

Slides say: “ k should be odd if the number of classes is even, and even if the number of classes is odd.” **More precisely:** an odd k prevents ties only for **two** classes. With three or more, no parity choice prevents ties — the slides’ own seven-point, four-class example still ties 2–2 at $k = 5$. The reliable tools are the tie-break (shrink k) and distance weighting (11.7).



Understand: ties come from the vote, not the distances. **Memorize:** classes+1, shrink on tie, $\sqrt{n}$, elbow. **Practice:** re-run the four-point experiment.

11.6 Bias, variance, and the decision boundary

Exam priority: Medium — the small- k /large- k table is in the slides; the boundary comparison is named in the syllabus.

k	Behaviour	Diagnosis
Very small (1)	The query takes the label of one nearby point, noise included	High variance — overfitting
Very large	Every query gets the dataset's majority class	High bias — underfitting
Moderate	Neither extreme	the good trade-off

The k -NN boundary is **irregular and locally determined** — the set of points where the vote flips. Small k : jagged and complex; large k : smooth and simple. It is **non-parametric**; its complexity grows with the data.

Classifier	Decision boundary
Decision tree	Axis-parallel rectangles — a staircase
Logistic regression	A single line or hyperplane ; a curve only with polynomial features
k-NN	Irregular, data-following ; jagged for small k , smooth for large k

[Supplementary — not examinable] The three-way comparison is assembled from three separate decks; no single slide places them side by side.

Understand: k is the complexity knob, like tree depth. **Memorize:** small k overfits, large k underfits; the three shapes. **Practice:** sketch boundaries for $k = 1$ and $k = 15$ on the same data.

11.7 Distance-weighted k-NN

Exam priority: High — the slides work a five-neighbour example in which weighting reverses the answer; that reversal is a favourite question.

Plain k -NN counts every one of the k neighbours equally, so two far-away green points can outvote one very close red point — contradicting k -NN's own inductive bias. **Weight each neighbour by closeness** and predict the class with the **largest total support**:

$$w_i = \frac{1}{d(x_q, x_i)^m} \quad \hat{f}(x_q) = \arg \max_{v \in V} \sum_{i=1}^k w_i \cdot \delta(v, f(x_i))$$

m sets how hard distance is penalized: $m = 1$ gives $1/d$, $m = 2$ gives $1/d^2$.

Worked example, $k = 5$: C 2.1 red, A 1.8 red, B 2.0 red, E 1.1 blue, D 0.7 blue.

Plain: 3 red to 2 blue gives **red**.

Weighted, $m = 1$:

$$\text{red} = \frac{1}{2.1} + \frac{1}{1.8} + \frac{1}{2.0} = 0.476 + 0.556 + 0.500 = 1.532$$
$$\text{blue} = \frac{1}{1.1} + \frac{1}{0.7} = 0.909 + 1.429 = 2.338$$

Blue wins, 2.338 to 1.532. **Weighting reversed the answer:** the two blue points, though fewer, are much closer.

Understand: the vote counted in proportion to nearness. **Memorize:** $w_i = 1/d^m$ and the support sum. **Practice:** redo with $m = 2$ and confirm blue still wins.

11.8 Feature scaling and non-numeric attributes

Exam priority: Medium — the income/age numbers and the encoding table are both in the slides.

Why scale. Person 1 (income 100000, age 30), person 2 (80000, 25):

$$d = \sqrt{(100000 - 80000)^2 + (30 - 25)^2} = \sqrt{400000000 + 25} \approx 20000$$

Income dominates completely — the age difference contributed 25 out of 400,000,025. After standardizing each feature to mean 0 and standard deviation 1, the slides' distance is about 1.14 and **both features count similarly**.

Method	Formula	Result
Min-max normalization	$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$	in $[0, 1]$
Standardization (Z-score)	$x' = \frac{x - \mu}{\sigma}$	mean 0, std 1

Encoding. k-NN needs numbers.

Case	Method	Example
Boolean	0 / 1	Male / Female
Ordered categories	integers in order	High school, College, MS, PhD become 1, 2, 3, 4
Unordered, more than two	one-hot	Cat, Dog, Zebra become (1,0,0), (0,1,0), (0,0,1)

The slides encode Education (ordered) as 1, 2, 3; Place — Bangalore, Udupi, Mandya — as two-bit codes (0,0), (0,1), (1,0); Gender as 0/1.

Why one-hot for Place? Numbering the cities 1, 2, 3 would tell the distance formula that Udupi lies “between” the other two. No such order exists; one-hot makes all three equidistant.

Understand: distance means something only when axes are comparable. **Memorize:** both scaling formulas and the three encoding cases. **Practice:** one-hot encode a four-value attribute.

11.9 The curse of dimensionality; pros and cons

Exam priority: Medium — the unit-cube derivation and its table are on the slides, and the pros/cons list is a standard short question.

k-NN assumes **similar points share labels**, but **in high dimensions randomly drawn points are almost never close**.

Unit-cube argument. Sample n points uniformly in $[0, 1]^d$; let ℓ be the edge of the smallest hypercube holding a test point's k nearest neighbours. Its volume is about the fraction of points it holds:

$$\ell^d \approx \frac{k}{n} \quad \implies \quad \ell \approx \left(\frac{k}{n}\right)^{1/d}$$

With $n = 1000$, $k = 10$:

d	$\ell = 0.01^{1/d}$	Meaning
2	0.10	10% of each edge — genuinely local
10	0.63	63% of each edge
100	0.955	barely local
1000	0.995	essentially the whole space

For large d **almost the entire space is needed to find the 10 neighbours**, so they are no more similar to the query than any other points — the assumption breaks. **More data?** Forcing $\ell = 0.1$ needs $n = k/\ell^d = k \cdot 10^d$ — **exponential**; for $d > 100$ that exceeds the electrons in the universe.

Remedies: (1) **weight the attributes** in the distance — area and locality over colour for house prices; (2) **leave one attribute out at a time** and test, converging on the best feature set; (3) **dimensionality reduction** such as PCA (Unit 4).

Pros (slides): no training phase; new data slots in without retraining; only two parameters, k and the distance; classification and regression. **Cons:** prediction is expensive in time (distance to every stored point) and space (all data kept); scales poorly; sensitive to scaling; the curse of dimensionality.

Slide quiz. *Large k on a small dataset?* No — distant points outvote the nearest. *Noise?* **Increase k** to dilute any single noisy point. *True of k -NN:* it needs features on one scale, struggles with many inputs, and assumes no functional form.

Understand: the volume argument, not just the table. **Memorize:** $\ell \approx (k/n)^{1/d}$, the remedies, the pros and cons. **Practice:** compute ℓ for $d = 5$.

12. Performance Metrics

12.1 The confusion matrix

Exam priority: Medium — every metric question starts by building it; the slides give one warm-up problem.

Given a classifier and an instance there are **four possible outcomes**, laid out in the **confusion matrix** (contingency table).

The reading rule: the second word, **Positive/Negative**, is the **PREDICTION**; the first, **True/False**, says whether it was **RIGHT**. A **False Positive** is a wrong prediction of Positive — the instance was negative.

		PREDICTED		
		Positive	Negative	
ACTUAL	Positive	TP	FN	<- all ACTUAL +
	Negative	FP	TN	<- all ACTUAL -
		^ all PREDICTED +		

Rows are actual counts, **columns** predicted counts, the **diagonal** everything the model got right.

Warm-up (slides). 200 objects, 100 A and 100 B; the model correctly predicted 60 A and 30 B. TP = 60, TN = 30; of the 100 real A's only 60 were caught so FN = 40; of the 100 real B's only 30 so FP = 70. **Correct 90, wrong 110.**

Understand: rows are truth, columns are prediction. **Memorize:** the reading rule. **Practice:** fill the matrix from a word problem.

12.2 Accuracy and the class-imbalance trap

Exam priority: High — the fraud example is worked in the slides, and “why accuracy fails on imbalanced data” is a standard question.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

A good measure only when the classes are nearly balanced.

The fraud example. 990 normal transactions and 10 fraudulent in training. On 100 test transactions a **naive program that calls everything normal** gets 98 right:

$$\text{Accuracy} = \frac{98 + 0}{100} = 98\%$$

— and catches **zero** frauds. **Accuracy is not a good measure when the dataset is imbalanced.**

Slides say: normal is the positive class and fraud the negative, and the matrix is drawn that way.
More precisely: the argument works either way, but the convention used everywhere else in this section — and in the recall discussion — makes the **rare, important class (fraud) the positive class**. Fix a convention before filling the matrix.

Understand: accuracy rewards guessing the majority. **Memorize:** the formula and the 98% example.
Practice: invent a matrix with high accuracy and zero recall.

12.3 Precision, recall, F1, specificity, and the full term list

Exam priority: Medium — the definitions, the “which to optimize” rule and the harmonic-mean reason are asked; the slides present the term list as one table to memorize.

Precision: of everything flagged positive, how many really were? Precision = $\frac{TP}{TP + FP}$

Matters when false positives are the expensive mistake — legitimate email sent to spam, an irrelevant product recommendation.

Recall (sensitivity, TPR, hit rate): of everything really positive, how many did we find? Recall = $\frac{TP}{TP + FN}$

Matters when false negatives are the expensive mistake — a missed cancer, a missed fraud.

Ask	Optimize
“Is a false alarm the expensive mistake?”	Precision — spam, recommendations
“Is a missed case the expensive mistake?”	Recall — cancer screening, fraud

F1 is the **harmonic mean** of the two — 0 worst, 1 best, reaching 1 **only when both are 100%**:

$$F_1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Why harmonic? It is dominated by the smaller value. Precision 1.0 with recall 0.0 has arithmetic mean 0.5 but $F_1 = 0$. One metric cannot be bought at the expense of the other.

Specificity (true negative rate): of everything really negative, how many did we call negative? Specificity = $\frac{TN}{TN + FP}$

Metric	Also called	Formula	Denominator
TPR	Recall, Sensitivity, Hit rate	$\frac{TP}{TP + FN}$	row — actual +
FNR	False negative rate	$\frac{FN}{TP + FN}$	row — actual +
TNR	Specificity	$\frac{TN}{TN + FP}$	row — actual –
FPR	1 – Specificity	$\frac{FP}{TN + FP}$	row — actual –
Precision	—	$\frac{TP}{TP + FP}$	column — predicted +
Accuracy	—	$\frac{TP + TN}{\text{all}}$	everything
F1	—	harmonic mean of P and R	—

Identities: $TPR + FNR = 1$, $TNR + FPR = 1$.

The denominator trick. Row-based metrics (recall, FNR, specificity, FPR) use one actual class and do not move when the class ratio changes. Precision divides by a column, mixing both classes. Hence the slides: **any metric using values from both classes — accuracy, precision, F-score — is inherently sensitive to class skew.** Section 13.2 uses this.

Understand: precision looks down a column, recall along a row. **Memorize:** all seven rows, the identities, spam-versus-cancer. **Practice:** cover the formula column and reproduce it.

12.4 Worked example — binary classifier

Exam priority: Medium — the $n = 165$ matrix is worked in the slides; the five-metric numerical is routine once 12.3 is known.

		PREDICTED		
		YES	NO	
ACTUAL	YES	TP=100	FN=5	= 105 actual yes
	NO	FP=10	TN=50	= 60 actual no
		110	55	n = 165

$$\text{Accuracy} = \frac{100 + 50}{165} = \frac{150}{165} = 0.909 \quad \text{Precision} = \frac{100}{110} = 0.909$$

$$\text{Recall} = \frac{100}{105} = 0.952 \quad \text{Specificity} = \frac{50}{60} = 0.833$$

$$F_1 = \frac{2 \times 0.952 \times 0.909}{0.952 + 0.909} = \frac{1.731}{1.861} = 0.930 \quad \text{FPR} = \frac{10}{60} = 0.167 = 1 - 0.833$$

Understand: every number comes from one of four cells. **Memorize:** nothing new. **Practice:** redo it with the cells permuted.

12.5 The multi-class confusion matrix, with a worked example

Exam priority: High — the one-vs-rest rule and the four-class matrix are both worked in the slides.

One row and one column per class. **For class X, treat X as positive and everything else as negative:** TP is X's diagonal cell; FN is the **rest of row X**; FP is the **rest of column X**; TN is everything outside row X and column X. For breeds Husky, Labrador, Bulldog: Husky's TP is $P(H, H)$, FN is $P(H, L) + P(H, B)$, FP is $P(L, H) + P(B, H)$, TN the other four cells. **Overall accuracy** is the diagonal over the total; **per-class precision** divides by the **column** sum, **per-class recall** by the **row** sum.

Worked example.

	Pred A	Pred B	Pred C	Pred D	Row total
Actual A	100	80	10	10	200
Actual B	0	9	0	1	10
Actual C	0	1	8	1	10
Actual D	0	1	0	9	10
Col total	100	91	18	21	230

$$\text{Accuracy} = \frac{100 + 9 + 8 + 9}{230} = \frac{126}{230} = 0.548$$

Class	TP	FP (rest of column)	FN (rest of row)	Precision	Recall
A	100	0	$80 + 10 + 10 = 100$	$100/100 = 1.000$	$100/200 = 0.500$
B	9	$80 + 1 + 1 = 82$	$0 + 0 + 1 = 1$	$9/91 = 0.099$	$9/10 = 0.900$
C	8	$10 + 0 + 0 = 10$	$0 + 1 + 1 = 2$	$8/18 = 0.444$	$8/10 = 0.800$
D	9	$10 + 1 + 1 = 12$	$0 + 1 + 0 = 1$	$9/21 = 0.429$	$9/10 = 0.900$

Macro-averages:

$$\text{Precision} = \frac{1.000 + 0.099 + 0.444 + 0.429}{4} = 0.493 \quad \text{Recall} = \frac{0.5 + 0.9 + 0.8 + 0.9}{4} = 0.775$$

$$F_1 = \frac{2 \times 0.493 \times 0.775}{0.493 + 0.775} = \frac{0.764}{1.268} = 0.603$$

What the numbers reveal. Class A has perfect precision but recall 0.5 — the model never wrongly calls something A, but dumps 80 real A's into B. Class B therefore has excellent recall and terrible precision. That is the precision–recall tension in numbers; accuracy alone (0.548) hides it.

Understand: row for FN, column for FP. **Memorize:** the one-vs-rest rule. **Practice:** compute class C's specificity from the matrix.

13. ROC and AUC

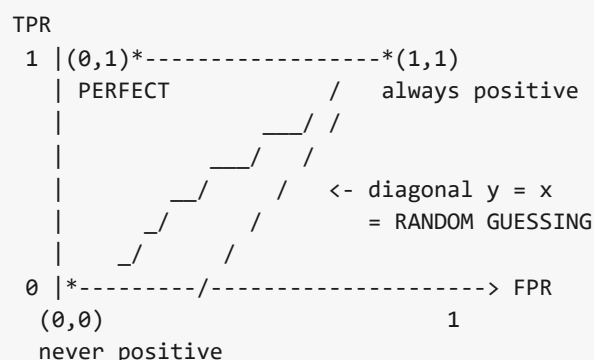
13.1 The ROC curve, ROC space, and a worked curve

Exam priority: Medium — the definition, the five special points and the obesity table are all in the slides; this is the theory-plus-plot half of the ROC question.

ROC (Receiver Operating Characteristics) plots TPR against FPR at various threshold values, separating the “signal” from the “noise”. It depicts the trade-off between benefits (true positives) and costs (false positives).

Axes: y is TPR (recall), x is FPR ($1 - \text{specificity}$).

The key insight. A probabilistic classifier needs a **threshold**, and **every threshold gives a different confusion matrix, hence a different (FPR, TPR) point**. The curve is the trail of those points as the threshold sweeps from 1 to 0; **lowering** it calls more instances positive, raising TPR and FPR together.



Point	Meaning
(0, 0)	Never predict positive — no false positives, no true positives
(1, 1)	Always predict positive
(0, 1)	Perfect classification
Diagonal $y = x$	Random guessing
Below the diagonal	Worse than random

A point is better than another if it lies to its NORTHWEST — higher TPR, lower FPR, or both.

Conservative classifiers (lower-left) have low TPR and low FPR — they demand **strong evidence** before predicting positive. **Liberal** classifiers (upper-right) predict positive on **weaker evidence**, with high TPR and high FPR.

Worked example — the obesity classifier.

Threshold	FPR	TPR
0 (everyone called obese)	1	1
0.3	0.75	1
0.4	0.5	1
0.6	0.25	0.75
0.7	0	0.75
0.9	0	0.5



The graph summarizes every confusion matrix the thresholds produced. Depending on how many false positives we accept, the optimal point is **(0, 0.75)** or **(0.5, 1)**: at threshold 0.7, 75% of positives with **zero** false alarms — choose it when false positives are costly; at threshold 0.4, **every** positive at a 50% false-positive rate — choose it when a miss is unacceptable, as in medical screening.

Understand: one classifier, many thresholds, one curve; each table row is one confusion matrix.

Memorize: the five points, northwest-is-better, conservative versus liberal. **Practice:** plot the six points and justify an operating point.

13.2 AUC — area under the ROC curve

Exam priority: Medium — the interpretation table is bookwork; the class-skew statement needs the correction below.

AUC measures how well the model distinguishes positives from negatives; the higher, the better. It compares models: the curve with the larger area is the better method.

AUC	Interpretation
1	Perfect separation
0.5 to 1	A high chance of distinguishing the classes
0.5	Cannot distinguish — a random or constant class
0	Every prediction inverted

AUC = 0 is perfectly *informative* — flip every prediction and you have AUC = 1. The useless model is AUC = 0.5.

[Supplementary — not examinable] AUC equals the probability that a random positive is ranked above a random negative.

ROC and class imbalance. TPR comes from the actual-positive row and FPR from the actual-negative row; neither mixes the classes, so the curve does not move when the class ratio changes with the classifier fixed. In the 990:10 fraud example the “always positive” classifier sits at (1, 1) on the diagonal with **AUC = 0.5** — exposed as worthless despite 98% accuracy.

Slides say: ROC/AUC is “insensitive to class skew”. **More precisely:** it is **much less affected than accuracy, precision or F1**, because it uses only row-based rates. It is not immune: with very few positives the TPR is estimated from a handful of cases and the curve is noisy, and AUC can look healthy while precision at any usable threshold is dreadful. Write “less affected than accuracy”, not “insensitive”.

Understand: why row-based rates resist skew, and where that ends. **Memorize:** the four-row table. **Practice:** explain why the naive fraud classifier scores 0.5.

14. Practice set

Ten questions in exam format. Attempt each on paper before reading the answer.

Q1 (4 marks). Define a well-posed learning problem. Write T, P and E for a system that learns to recommend movies to a user.

Answer. A well-posed learning problem is one in which a program improves its performance P at task T with experience E (Mitchell). Movies: **T** — recommend movies the user will watch and rate highly; **P** — the fraction of recommendations the user watches, or their mean rating; **E** — the user’s watch-and-rating history plus other users’ histories. T is an action, P a number, E data.

Q2 (8 marks). An instance is described by four attributes, each with three values. (a) How many distinct instances are there? (b) How many concepts? (c) How many semantically distinct conjunctive hypotheses? (d) Apply Find-S to the data below, showing the hypothesis after every row.

Experience	Location	Tech Stack	Team Size	Remote
Junior	City	Frontend	Small	No
Senior	Suburb	Fullstack	Medium	Yes
Senior	City	Fullstack	Large	Yes
Mid	Suburb	Backend	Small	No
Senior	Rural	Fullstack	Medium	Yes

Answer. (a) $|\mathcal{X}| = 3 \times 3 \times 3 \times 3 = 81$. (b) Each instance can be labelled two ways, so 2^{81} concepts. (c) $|H| = (3 + 1)^4 + 1 = 256 + 1 = 257$. (d) Trace:

Step	Row	Action	h
0	—	initialize	$\langle \emptyset, \emptyset, \emptyset, \emptyset \rangle$
1	1	negative, ignore	unchanged
2	2	first positive, copy	$\langle \text{Senior, Suburb, Fullstack, Medium} \rangle$
3	3	Location and Team Size differ, generalize	$\langle \text{Senior, ?, Fullstack, ?} \rangle$
4	4	negative, ignore	unchanged
5	5	already covered	$\langle \text{Senior, ?, Fullstack, ?} \rangle$

Final hypothesis: **remote work is offered to Senior Fullstack developers regardless of location or team size**. Check against the negatives: row 1 is Junior and row 4 is Mid, so both are predicted negative — consistent.

Q3 (10 marks). Build the ID3 decision tree for the data below. Show every entropy and gain.

#	Weather	Traffic	Day	GoOut
1	Sunny	Light	Weekday	Yes
2	Sunny	Heavy	Weekend	Yes
3	Sunny	Heavy	Weekday	Yes
4	Rainy	Light	Weekend	Yes
5	Rainy	Heavy	Weekday	No
6	Rainy	Heavy	Weekend	No
7	Rainy	Heavy	Weekday	No
8	Sunny	Light	Weekend	Yes

Answer. $|S| = 8$, Yes 5, No 3.

$$\text{Entropy}(S) = -\left(\frac{5}{8} \log_2 \frac{5}{8} + \frac{3}{8} \log_2 \frac{3}{8}\right) = 0.424 + 0.531 = 0.954$$

Weather. Sunny = rows 1,2,3,8, all Yes, entropy 0. Rainy = rows 4,5,6,7, 1 Yes : 3 No, entropy 0.811.

$$I = \frac{4}{8}(0) + \frac{4}{8}(0.811) = 0.406 \quad \text{Gain} = 0.954 - 0.406 = \mathbf{0.548}$$

Traffic. Light = rows 1,4,8, all Yes, entropy 0. Heavy = rows 2,3,5,6,7, 2 Yes : 3 No, entropy 0.971.

$$I = \frac{3}{8}(0) + \frac{5}{8}(0.971) = 0.607 \quad \text{Gain} = 0.954 - 0.607 = \mathbf{0.347}$$

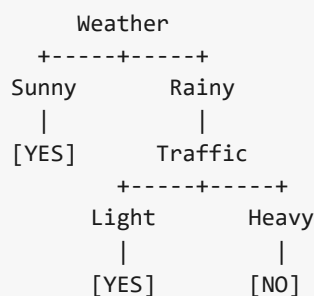
Day. Weekday = rows 1,3,5,7, 2 : 2, entropy 1. Weekend = rows 2,4,6,8, 3 : 1, entropy 0.811.

$$I = \frac{4}{8}(1) + \frac{4}{8}(0.811) = 0.906 \quad \text{Gain} = 0.954 - 0.906 = \mathbf{0.048}$$

Root = Weather. Sunny is pure, so it is a leaf **Yes**. Rainy (rows 4,5,6,7; entropy 0.811) needs another split.

Traffic under Rainy. Light = row 4, Yes, entropy 0. Heavy = rows 5,6,7, all No, entropy 0. $I = 0$, so **Gain = 0.811**. *Day under Rainy.* Weekday = rows 5,7, both No, entropy 0. Weekend = rows 4,6, 1:1, entropy 1. $I = \frac{2}{4}(0) + \frac{2}{4}(1) = 0.5$, Gain = 0.311.

Traffic wins and both of its branches are pure.



Day never appears — it carried no useful information.

Q4 (6 marks). Four models are trained on the same data. Diagnose each as high bias, high variance, both, or neither, assuming a Bayes error near 0%. Then say how the diagnosis of Model B changes if the Bayes error is known to be 12%.

	A	B	C	D
Train error	2%	13%	14%	1%
Test error	14%	14%	28%	2%

Answer. Using the proxy rule — training error indicates bias, the train-to-test gap indicates variance: - **A:** train 2% (low bias), gap 12 points — **high variance**. - **B:** train 13% (high bias), gap 1 point — **high bias**. - **C:** train 14% and a 14-point gap — **high bias and high variance**. - **D:** train 1%, gap 1 point — **neither; the model we want**.

With a Bayes error of 12%, Model B's 13% training error is only 1 point above the best achievable, so it is **not high bias** — it is near the theoretical limit, and its 1-point gap means low variance too. The same numbers give a different diagnosis because “high” training error is measured relative to the Bayes error, not to zero.

Q5 (8 marks). The true function is $f(x) = 3x$. At the test point $x_0 = 1$, three models trained on three different datasets predict 2.6, 3.2 and 2.9. The noise has variance 0.25. Compute bias-squared, variance, and the total expected error.

Answer. True value $f(1) = 3$.

Average prediction: $(2.6 + 3.2 + 2.9)/3 = 8.7/3 = 2.9$.

Bias = $2.9 - 3 = -0.1$, so $\text{Bias}^2 = 0.01$.

Variance: deviations from 2.9 are $-0.3, +0.3, 0$; squares 0.09, 0.09, 0; sum 0.18; divide by 3: **0.06**.

Irreducible error $\sigma^2 = 0.25$.

Total = $0.01 + 0.06 + 0.25 = \mathbf{0.32}$. The irreducible term dominates; the model's own error is 0.07.

Q6 (6 marks). A logistic regression classifier has $\theta = [-6, 2, 1]$ over features (x_1, x_2) . (a) Write the decision boundary. (b) Classify the points $(2, 1)$ and $(3, 2)$ and give $P(y = 1 | x)$ for each.

Answer. (a) Predict 1 when $\theta^T x \geq 0$, that is, $-6 + 2x_1 + x_2 \geq 0$. **The boundary is the line $2x_1 + x_2 = 6$.** (b) For $(2, 1)$: $z = -6 + 4 + 1 = -1 < 0$, so **class 0**; $h = 1/(1 + e^1) = 1/(1 + 2.718) = 1/3.718 = 0.269$. For $(3, 2)$: $z = -6 + 6 + 2 = 2 \geq 0$, so **class 1**; $h = 1/(1 + e^{-2}) = 1/(1 + 0.135) = 1/1.135 = 0.881$.

Q7 (8 marks). Using the medical table of Section 11.3 and the same query (100, 135, 12, 8), classify the patient with $k = 3$ using **Manhattan** distance. Then apply distance-weighted k-NN with $w = 1/d$ on the same three neighbours.

Answer. Manhattan distances: - A: $|0| + |15| + |0| + |2| = 17$ - B: $10 + 5 + 2 + 3 = 20$ - C: $20 + 25 + 1 + 1 = 47$ - D: $0 + 5 + 1 + 1 = 7$ - E: $15 + 5 + 1 + 2 = 23$

Three nearest: D (7, No), A (17, No), B (20, Yes). Mode is **No**.

Weighted: $\text{support}(\text{No}) = 1/7 + 1/17 = 0.1429 + 0.0588 = 0.2017$; $\text{support}(\text{Yes}) = 1/20 = 0.05$. **No** again, and by a wider margin — D is by far the closest point.

Q8 (6 marks). A continuous attribute Age has the sorted values 22, 30, 35, 40, 45, 50, 55 with classes N, Y, Y, Y, Y, N, N. Find the best threshold.

Answer. $|S| = 7$, 4 Y : 3 N, $\text{Entropy}(S) = -(\frac{4}{7} \log_2 \frac{4}{7} + \frac{3}{7} \log_2 \frac{3}{7}) = 0.461 + 0.524 = 0.985$.

Class changes between 22 and 30, and between 45 and 50, so the candidates are $c_1 = (22 + 30)/2 = 26$ and $c_2 = (45 + 50)/2 = 47.5$.

$c_1 = 26$: left {22}, pure, entropy 0; right six values, 4 Y : 2 N, entropy 0.918. $I = \frac{1}{7}(0) + \frac{6}{7}(0.918) = 0.787$. Gain = $0.985 - 0.787 = 0.198$.

$c_2 = 47.5$: left five values, 4 Y : 1 N, entropy $-(0.8 \log_2 0.8 + 0.2 \log_2 0.2) = 0.258 + 0.464 = 0.722$; right {50, 55}, pure, entropy 0. $I = \frac{5}{7}(0.722) + \frac{2}{7}(0) = 0.516$. Gain = $0.985 - 0.516 = 0.469$.

Threshold 47.5 wins (0.469 against 0.198). The tree tests whether Age is below 47.5.

Q9 (8 marks). (a) A binary classifier on 200 instances gives TP = 40, FN = 10, FP = 20, TN = 130. Compute accuracy, precision, recall, specificity, F1 and FPR. What would the accuracy and recall be if the model simply predicted every instance negative? (b) For the three-class matrix below, compute overall accuracy and the precision and recall of each class.

	Pred X	Pred Y	Pred Z
Actual X	50	5	5
Actual Y	10	30	0
Actual Z	0	10	40

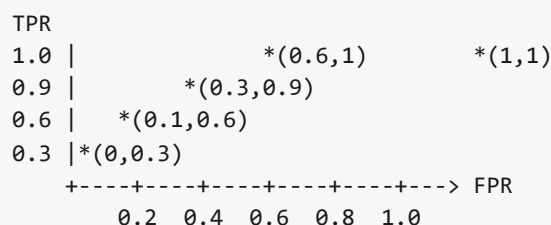
Answer. (a) Accuracy = $(40 + 130)/200 = 170/200 = 0.85$. Precision = $40/(40 + 20) = 40/60 = 0.667$. Recall = $40/(40 + 10) = 40/50 = 0.80$. Specificity = $130/(130 + 20) = 130/150 = 0.867$. $F_1 = 2(0.667)(0.80)/(0.667 + 0.80) = 1.067/1.467 = 0.727$. FPR = $20/150 = 0.133 = 1 - 0.867$. All-negative model: TN = 150, FN = 50, so accuracy = $150/200 = 0.75$ and recall = $0/50 = 0$. Accuracy barely drops while recall collapses — the imbalance trap.

b. Row totals 60, 40, 50; column totals 60, 45, 45; grand total 150. Accuracy = $(50 + 30 + 40)/150 = 120/150 = 0.80$.

- X: precision = $50/60 = 0.833$; recall = $50/60 = 0.833$.
- Y: precision = $30/45 = 0.667$; recall = $30/40 = 0.75$.
- Z: precision = $40/45 = 0.889$; recall = $40/50 = 0.80$.

Q10 (6 marks). A classifier evaluated at five thresholds gives the (FPR, TPR) pairs (1, 1), (0.6, 1), (0.3, 0.9), (0.1, 0.6), (0, 0.3). Plot the ROC curve, identify the conservative and liberal ends, choose an operating point for a disease-screening application, and explain why a classifier that always predicts positive has AUC 0.5.

Answer.



The lower-left points, (0, 0.3) and (0.1, 0.6), are the **conservative** end — few false alarms, many misses. The upper-right, (0.6, 1) and (1, 1), are the **liberal** end. For disease screening a miss is the expensive error, so choose a high-recall point: **(0.3, 0.9)** catches 90% of cases at a 30% false-alarm

rate, or (0.6, 1) if every case must be caught and follow-up tests are cheap. The always-positive classifier sits at (1, 1); the curve through (0, 0) and (1, 1) is the diagonal, whose area is 0.5 — it has no ability to rank positives above negatives, which is what AUC measures.

15. Formula sheet

Concept learning

Quantity	Formula
Feature space	$ \chi = \prod_i A_i $ ($= 2^n$ if all binary)
Concept space	$2^{ \chi }$
Hypothesis space, conjunctive	$ H = \prod_i (A_i + 1)$
Version space	$VS = \{h \in H : h \text{ consistent with } D_{\text{train}}\}$

Concept space contains hypothesis space contains version space. Bias shrinks the first; data shrinks the second. **Find-S**: restriction bias from its search space. **ID3**: preference bias (shortest tree, high gain near the root) from its search strategy.

Bias–variance

$$\mathbb{E}[(y - \hat{f}(x))^2] = (\mathbb{E}[\hat{f}(x)] - f(x))^2 + \mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2] + \sigma^2$$

Overfit: low bias, high variance, low train error, big train-test gap. Underfit: high bias, low variance, high train error, small gap. Proxy rule: train error indicates bias; the gap indicates variance — valid only for near-zero Bayes error.

Decision trees

$$\text{Entropy}(S) = - \sum_{i=1}^c p_i \log_2 p_i \quad (\max = \log_2 c; 0 \log_2 0 = 0)$$

$$I(A) = \sum_v \frac{|S_v|}{|S|} \text{Entropy}(S_v) \quad \text{Gain}(S, A) = \text{Entropy}(S) - I(A)$$

Pick the maximum gain. Quick entropies: pure 0, 1:1 = 1, 1:2 = 0.918, 1:3 = 0.811, 2:3 = 0.971, 3:4 = 0.985, 1:6 = 0.592, 5:9 = 0.940. Continuous attribute: sort, midpoints where the class changes, max gain wins. Post-pruning beats pre-pruning; train / validation / test.

Logistic regression

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}} = P(y = 1 \mid x; \theta) \quad \text{boundary: } \theta^T x = 0$$

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\theta}(x^{(i)})) \right]$$

$$\theta_j := \theta_j - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} \quad (\text{all } \theta_j \text{ simultaneously})$$

Squared error is non-convex here. The update looks like linear regression's; only h_{θ} differs.

k-NN

$$d(x, y) = \left(\sum_{i=1}^d |x_i - y_i|^q \right)^{1/q} \quad q = 1 \text{ Manhattan, } q = 2 \text{ Euclidean}$$

Classification: mode of the k labels. Regression: mean. Weighted: $w_i = 1/d(x_q, x_i)^m$, largest weight sum wins. Choosing k : classes + 1, shrink on a tie, $\sqrt{n}$, elbow. Small k overfits, large k underfits. Curse: $\ell \approx (k/n)^{1/d}$; $\ell = 0.1$ needs $n = k \cdot 10^d$. Scaling: min-max $(x - x_{\min})/(x_{\max} - x_{\min})$; z-score $(x - \mu)/\sigma$.

Performance metrics

Metric	Formula	Denominator
Accuracy	$\frac{TP + TN}{TP + TN + FP + FN}$	everything
Precision	$\frac{TP}{TP + FP}$	column, predicted +
Recall / Sensitivity / TPR	$\frac{TP}{TP + FN}$	row, actual +
Specificity / TNR	$\frac{TN}{TN + FP}$	row, actual -
FPR = 1 - Specificity	$\frac{FP}{TN + FP}$	row, actual -
FNR = 1 - Recall	$\frac{FN}{TP + FN}$	row, actual +
F1	$\frac{2PR}{P + R}$	harmonic mean

Multi-class, class X: TP diagonal, FN rest of row X, FP rest of column X, TN the rest. Accuracy fails on imbalance. Precision when false alarms are costly; recall when misses are costly.

ROC / AUC

Axes TPR (y) against FPR (x), one point per threshold. (0,0) never positive; (1,1) always positive; (0,1) perfect; diagonal random; below it worse than random; northwest is better. AUC 1 perfect, 0.5 random or constant, 0 inverted. ROC is less affected by class skew than accuracy, because both rates are row-based.

16. Most likely exam questions

1. **Trace ID3 on a given table** — full entropy, $I(A)$ and gain for every attribute, then recurse. Reason: the largest worked example in the slides, repeated in the lab.
2. **Compute $|\chi|$, the concept space and $|H|$** for mixed-arity attributes. Reason: a worked six-attribute numerical in the slides with an easily varied setup.
3. **Trace Find-S, writing h after every row, and list its limitations.** Reason: two solved practice problems in the deck and an explicit limitations slide.
4. **k-NN classification numerical** — distance table, sort, mode — and the **weighted variant that reverses the answer.** Reason: three worked k-NN problems in the slides and a lab on the same.
5. **Compute every metric from a confusion matrix, binary and multi-class.** Reason: the $n = 165$ and four-class matrices are both worked in full.
6. **Explain bias and variance, diagnose from train/test error, and do the decomposition numerical.** Reason: the four-model table and the three-dataset numerical are both on the slides.
7. **State the formal definition of overfitting and compare the pruning methods.** Reason: the definition is presented for memorization and reduced error pruning is traced.
8. **Derive the logistic regression decision boundary from θ , and explain why squared error is not used.** Reason: both the linear and circular boundary are worked, and the non-convexity slide is explicit.
9. **Preference versus restriction bias — ID3 against Find-S.** Reason: the slides bold the search-strategy versus search-space sentence.
10. **Plot an ROC curve from a threshold table and interpret AUC values.** Reason: the obesity table is worked point by point and the AUC table is presented for memorization.

End of Unit 1 Master Notebook.